

An ASIC Design Flow for High-Level Synthesis of Asynchronous Circuits

Mayuresh Warunjikar, Madhav Desai

Abstract—Asynchronous circuits have some benefits over their synchronous counterparts, such as lower idle power consumption, potentially lower latency, absence of clock distribution issues, among others. However, industry-scale electronic design automation (EDA) support, particularly for high-level synthesis (HLS) of asynchronous systems from imperative programs, remains limited. We present an approach to synthesize asynchronous circuits from imperative programs using the AHIR HLS framework. The framework models the control path in the form of a restricted form of Petri nets that are 1-safe and live by construction. In this work, we start with this representation in AHIR and generate asynchronous circuits with a delay insensitive controller, a single-rail data path with bundled-data interaction between the controller and the data path that resembles the micropipelines architecture. We formalize the timing constraints required for correct functioning of the generated circuit and show that those constraints are one sided (feasible), necessary and sufficient. We have also developed a static timing analyzer to check that the circuit meets these constraints. Our flow integrates seamlessly with off-the-shelf ASIC toolchains and standard synchronous cell libraries. To demonstrate our approach, we show an end to end implementation of an AES encryption algorithm and use post layout circuit simulation to verify the correctness and estimate the performance of the resultant circuit.

Index Terms—VLSI Design, High-Level Synthesis (HLS), Electronic Design Automation (EDA), Asynchronous Circuits

I. INTRODUCTION

A. About asynchronous circuits

Synchronous circuits have long been the dominant paradigm in VLSI design. In synchronous circuits, state transitions happen at discrete time steps, synchronized by a global clock signal. Asynchronous circuits, instead, are characterized by absence of a clock and state transitions happen using control signal handshakes between communicating components. A synchronous circuit design assumes that the clock signal reaches all the sequential elements synchronously. With VLSI technology scaling, the gate delays reducing and wire delays becoming significant, meeting this assumption is more challenging due to clock skew issue [1]. Distributing the clock with sufficient signal strength also puts a significant drain on the power source [2], [3]. In situations where the devices have to spend a larger part of their life in waiting for an external event, such as in edge devices, this significantly reduces the energy efficiency of the devices.

Asynchronous circuits, preempt several of these issues by not having a globally distributed clock and in turn the notion

of clock skews. In idle states, asynchronous circuits, consume only the leakage power. In synchronous circuits, clock and power gating partly addresses this concern. However there are trade-offs involved between the idle power consumption and the wake-up latency [4] and the power supply still needs to be designed for the peak clock load. Low idle power consumption is natural in asynchronous circuits without complex clock and power gating scheme and wake-up latency trade off. With energy efficiency becoming a significant objective for sustainable computing [5], [6], these aspects make asynchronous circuits an attractive option for VLSI designers.

Besides, asynchronous circuits have lower electromagnetic interference due to absence of concentrated switching at a single frequency that the synchronous circuits have [7]. A related and active area of research in the security domain is whether this makes the side-channel attacks harder [8]–[11].

Asynchronous circuits also permit a more modular development of hardware components. In synchronous circuits the timing constraints are global in nature. The clock frequency is limited by the slowest performing combinational logic component in the circuit. In asynchronous circuits, it is possible to focus the optimization effort on most frequently used components since a slower component does not impose a time constraint on other components.

This also implies that, in synchronous circuits the latency is a function of the worst case combinational delay, while in asynchronous case it is that of the average case delay, which potentially results in reduction in overall latency.

However, asynchronous circuits do have some challenges. Unlike synchronous circuits, asynchronous circuits must operate correctly in continuous time, avoiding spurious transitions of signals, called glitches or hazards. Hence these circuits are also more sensitive to crosstalk. With continuous time state changes, formal verification methods face a greater state space explosion. In absence of a central synchronizing signal (clock), handshake logic is required between any two communicating components, which adds to the area and latency. Detection of completion of a computation needs to be explicit. Approaches like dual-rail design address this but add to significant energy and area costs. Historically, the EDA tool support for asynchronous circuits is thin. No asynchronous aware placement and routing tools, no standards (e.g. to express timing constraints) have evolved for asynchronous systems. There are only a few academic HLS frameworks. Standard cell libraries have evolved only for synchronous designs. Testing methods, too, are focused on synchronous designs.

There are several application areas, where the energy efficiency and low idle power consumption are highly de-

sirable where asynchronous circuits may find applications. There are several studies that show that the battery life in Implantable Medical Devices (IMD) is the primary concern given the surgical intervention it needs to replace a battery [12]–[15]. Environmental Wireless Sensor Networks (EWSN) have similar challenges in replacement of batteries and stricter energy budgets, besides large idle times [16]. There are several use cases of edge devices that have to work with energy constraints, with event driven bursts of activity followed by large idle times, where asynchronous circuits may be useful. For instance, event cameras, which asynchronously send pixel data when the brightness changes, have sparse asynchronous activity, low energy budget and low latency requirement [17].

B. Our approach

The goals of this exploration are: 1) To produce a flow from HLS design entry to physical layout of an asynchronous integrated circuit, leveraging off-the-shelf flows and cell libraries. 2) To prove that the flow produces a circuit that faithfully implements the design entry. 3) Through experiments, assess the feasibility of the flow and the performance of generated circuits on various dimensions. 4) Assess the class of applications suitable for such methodology.

To this effect, the approach of this exploration, in brief, is: 1) AHIR is an HLS framework [18] that accepts design entry in the form of an imperative language and converts it into an intermediate representation called virtual circuit. Further, it synthesizes synchronous circuit RTL from the virtual circuits. Our approach to synthesis of asynchronous circuits begins at the virtual circuit representation in AHIR. 2) The virtual circuit represents the control path using a restricted form of 1-safe and live Petri nets [19], a data path and the control - data linkages. Our asynchronous synthesis tool produces RTL code from the virtual circuits. We describe our synthesis approach in Section III. 3) We design the data path and the control - data interaction using a latch based design. We formulate the timing constraints for correct functioning of this circuit. To check if an implementation meets these constraints we have developed a static timing analysis tool using OpenSTA [20] tool as a library. We describe these aspects of the physical design in Section IV. 4) We prove the correctness of the synthesis approach by way of trace equivalence between the virtual circuits and the synthesized circuits, through many steps, including reasoning about hazards in the synthesized circuit in Section V. 5) One of our goals is to use off-the-shelf tools to the extent possible to implement asynchronous circuits. We use OpenLane [21] toolchain and SkyWater cell library [22] to take the synthesized design to a physical design and use our timing analysis tool to verify the timing constraints. We demonstrate the experimental results based on post-implementation timing simulations and various metrics gathered from a case study (128-bit AES encryption) in Section VII.

C. Related work

Asynchronous design methodologies are often characterized by the delay models they use. The delay models reflect

the assumptions about the gate and wire delays that the methodology makes.

Huffman [23] introduced one of the early delay models called *bounded-delay model*. The circuits built using this model are also known as *Huffman circuits*. These circuits operate in what is called the *fundamental mode*, which permits only one of the inputs to change at a time. Upper bound on the time the circuit takes to stabilize after an input changes is expected to be known. The gap between input changes has to be greater than this bound. While such fundamental mode assumption is very restrictive for a practical circuit, it is useful for reasoning individual components in a circuit.

Nowick et al. [24] described an approach called *Burst Mode* circuits that borrow some characteristics of synchronous circuits and some of Huffman circuits. In these circuits, multiple inputs are allowed to change in *bursts*. A local clock is used to hold any outputs that feed back to the combinational logic in a stable state while they compute the response to a burst of inputs.

Ivan Sutherland, in his Turing Award lecture [25], introduced *Micropipelines*, in the form of an elegant design for asynchronous elastic pipelines. The approach uses a *two-phase bundled-data* convention. In two phase signaling, also called *transition signaling*, a control signal event is said to occur when its logic level changes from either 0 to 1 or 1 to 0. In bundled-data approach the data flow is moderated by control signals such that the delays in data transmission are less than the delays in the control signal that signals the writing of that data to a state holding element.

Delay Insensitive (DI) circuits is the most robust model of asynchronous circuits, introduced by Molnar et al. [26] and formally described by Udding et al. [27]. It makes no assumptions about the delays of wires and gates in a circuit. But, Martin [28] made some profound observations that proved the class of circuits that can be implemented as entirely DI circuits is very limited. He introduced a class of circuits called *Quasi Delay Insensitive (QDI) circuits* by extending the DI circuits with a notion of *isochronic forks*. Variations such as *extended isochronic forks* by Berkel et al. [29], extend the idea further. Manohar and Martin [30] showed that QDI circuits are Turing-complete. Manohar and Moses [31] refined Martin's approach to cover a wider class of circuits.

Speed Independent circuits, first described by Miller [32], make an assumption that wire delays are negligible but the gate delays can be arbitrarily large. This approach is similar to QDI circuits, since the assumption about wire delays amounts to assuming that all forks are isochronic.

Signal Transition Graphs (STG) [33] is a methodology for specifying asynchronous circuits. STGs are essentially Petri nets in which rising and falling transitions on a wire are modeled as separate Petri net transitions. There are constraints on the Petri net, to ensure that the rising and falling transitions on a signal strictly alternate and they do so only after the successor transition of a given transition has taken place, to ensure the stability.

Globally Asynchronous, Locally Synchronous (GALS), a term first used by Chapiro [34], refers to a design pattern

that combines some benefits of both the synchronous and asynchronous paradigms.

Beerel [35] and Myers [36] have covered asynchronous circuits comprehensively in their books. Hauck [37], provides a good survey of various delay models and design methodologies in asynchronous circuits. Balsa [38] and Act [39] are some of the mature frameworks that provide a design flow for asynchronous circuits.

Our approach is largely inspired by the Micropipelines architecture. We model the asynchronous circuits with a delay insensitive controller and a bundled-data interaction between the control and the data path. But we cater to fairly arbitrary control flows derived from imperative programs using a restricted form of Petri nets. Unlike Micropipelines, our approach does not require cells specifically designed for asynchronous circuits; it works with off-the-shelf ASIC flows and cell libraries designed for synchronous circuits.

The notion of delay-insensitivity in our synthesis of controller circuits is in line with Martin’s insights on the limitations of delay-insensitivity. However, Martin’s formulation that models a circuit as a set of production rules, expressly excludes the use of arbiters or branching elements. We permit the use of branch elements in our formulation. Although the branch element itself is not a DI circuit, we show that its use in the controller does preserve the DI property of the controller circuit. This allows us to build controllers that are DI, yet model fairly arbitrary control flows from imperative programs.

Our approach does not involve the use of QDI model or isochronic forks, which have stringent equality constraint on timing delays on forked paths that re-converge. We only have relative, one sided timing constraints.

Like STGs, we, too, use Petri nets to model the control paths. However, STG model relates very closely to the circuit realization, while our Petri nets are more abstract. For instance, transitions in an STG Petri net model a rising or falling edge of a logic level in a circuit, while in our Petri nets, a transition corresponds to a logical event. Due to their abstract nature they can be used for synthesizing asynchronous as well as synchronous systems. Further, our Petri nets isomorphically map to the asynchronous circuits we synthesize. STGs on the other hand require solving the Complete State Coding (CSC) problem, involving global reachability analysis [33]. Our Petri nets, despite being restrictive, are easy to build from imperative programs, for which AHIR HLS provides a pathway.

Different mathematical formalisms are used to model and reason about asynchronous circuits by different researchers. For example, Martin’s work uses production rules approach, STGs use transition systems. We use *prime event structures* [40], a formalism based on modeling the causality and conflict relation between events and a mapping of 1-safe Petri nets to them, to reason about the behavior of our specification and the synthesized circuits. We find that that a *true concurrency* formalism like this is useful to model asynchronous circuits where state transitions are continuous, unlike the discrete transitions in synchronous systems.

Systems like Act and Balsa have matured over years. There is a certain overlap of goals of these systems and those of ours. Our emphasis is on use of an imperative language for

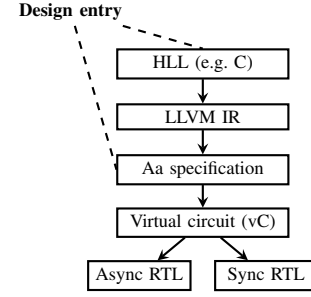


Fig. 1: AHIR HLS flow from design entry to RTL

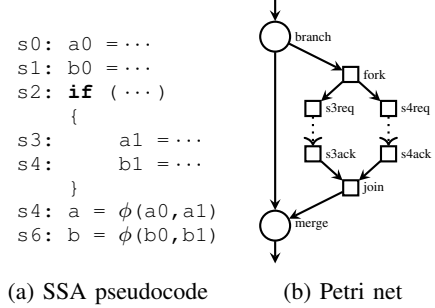


Fig. 2: An example of Petri net formation in AHIR from an SSA representation

design entry and on leveraging and being interoperable with off-the-shelf ASIC flows and cell libraries, primarily designed for synchronous systems. Also, our framework offers a choice of both synchronous and asynchronous paradigms without requiring any change in the design entry. This is particularly useful for exploratory and research work.

Some asynchronous circuit design methodologies use *dual-rail* data encoding as an alternative to the bundled-data approach. A dual rail system uses two wires to encode one bit of data to help detect completion of a computation. This helps in building a robust DI circuit where achieving timing closure is easy. However these benefits come for a significant area and energy costs. In present work, we choose the bundled-data method which saves on area and energy consumption. However tuning of the delays to meet the bundled-data constraints requires some manual steps which our static timing analysis tool assists in.

II. METHODOLOGY

A. AHIR flow from design entry to RTL

Fig. 1 shows the design flow in AHIR high-level synthesis framework. This flow for synchronous circuits is described in detail in [18]. The steps in the flow are: 1) AHIR has a design entry language named *Aa*, which is an imperative language that predominantly uses single assignment variables [41]. It has a notion of communication channels called *pipes* and a notion of *storage* which models the memory. The language has constructs to model pipelined flows in the form of loop statements. The language is fully described in [42]. 2) AHIR also accepts design entry in high-level languages, like C, that can be converted to an LLVM [43] intermediate representation (IR). With reasonable restrictions such as absence of

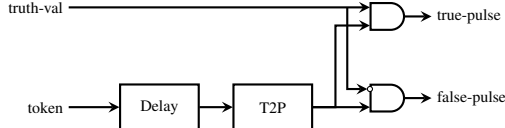


Fig. 3: Design of branch element

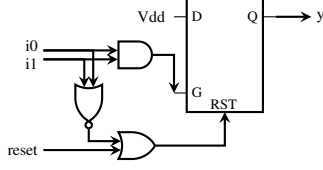


Fig. 4: Design of Muller C element

recursion and system calls. 3) The Aa specification is then converted into an intermediate representation called virtual circuit (vC), which also has a text form notation. The vC representation consists of the control path modeled as a Petri net, the data flow and the control-data interactions. 4) The vC representation is used to generate either synchronous or asynchronous RTL (in VHDL). The asynchronous synthesis process generates structural form VHDL code that instantiates components from our asynchronous RTL library.

Fig. 2 illustrates how the control flow aspect of an SSA form representation is modeled in a Petri net.¹, which shows four main aspects of a control flow - *fork*, *join*, *branch* and *merge*. The branches and merges modeled as places in Petri nets represent the conditional flows, while the forks and joins modeled as Petri net transitions represent the parallelism and synchronization. Each data path *operation* follows a *request-ack* protocol², which are modeled as Petri net transitions. The dotted arcs indicate the dependencies between the request and ack transitions that are realized by the data path operators.

B. Key components from asynchronous RTL library

We describe two key components from the RTL library. Section III will provide mapping of Petri nets to the RTL components. Section VI will describe some additional components.

Fig. 3 shows the design of a branch element that models a branch place, which uses an input from the data path to resolve the conflict. The T2P block converts a transition on the incoming Petri net arc into a pulse, which produces either a true-pulse or a false-pulse output, depending on the data path input. The transitions that succeed a branch place have a toggle flip flop, which flips its output value on receipt of the pulse, maintaining the transition signaling semantics. The delay element lets the truth value stabilize before interpreting it. In Section IV we will be formalizing the constraints on the delay value and ways to meet those constraints.

Fig. 4 shows a latch based design of a Muller C element that models a join transition. On system reset or when all inputs are at logic 0, the latch resets making the output logic 0. When

¹For brevity, we eliminate trivial (conduit) places and transitions.

²To keep the illustration simple, we show only one request-ack pair per operator, though there are two such pairs per operator in AHIR virtual circuits.

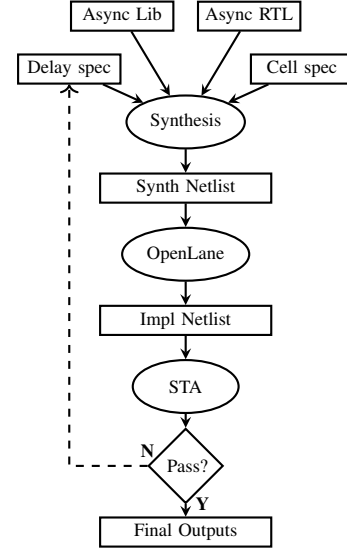


Fig. 5: ASIC flow for Asynchronous RTL generated

```
package vcdelays is
    constant nodedelays : t_delays_arr := (
        dp_1617 => (compute_delay => (3,0), others => (0,0)),
        ....
        others => ( others => ( 1, 0 ) )
    );
end package;
```

Fig. 6: An example of delay specification

all inputs are logic 1, the latch output becomes logic 1. When inputs are not in consensus, the output reflects the previously latched value. A hazard free operation requires that all the inputs change monotonically in one direction before any of them flips back, until the output changes. We will be proving this aspect in Section V.

C. AHIR asynchronous RTL to implementation flow

Fig. 5 shows the flow from RTL to the implementation. The generated RTL and the RTL library are fed to the synthesis process. The *Cell spec* identifies the cell types that are used as delay elements, which are buffers that are specially designed to produce a combinational delay in an area efficient way. *Delay spec* is a VHDL structure that is used to tune the delays inserted in the circuit to meet the timing constraints, illustrated in Fig. 6. The dashed arrow shows the iterative process of tuning the delay specification.

We use GHDL, an open source VHDL simulator, [44], GHDL-Yosys plugin [45] and Yosys synthesis tool [46] to convert these inputs into a synthesis netlist. We use the OpenLane [21] framework and SkyWater open source PDK [22] as the ASIC implementation flow. However, the flow can be easily adopted to work with other tools and PDKs. This flow produces an implementation netlist with placement, routing, parasitics information, SPICE model of the implementation etc.

The static timing analyzer (STA) in the flow, is developed by us for our asynchronous design methodology. We use the

```

Note: unit=ps
Note: SLACK=ACTUAL-REQD
Note: -ve SLACK is FAILURE
Note: +ve SLACK is room for optimization
TOPINST      DELAYTYP    ACTUAL      REQD      SLACK
dp_1617      compute_delay 1631.80  2088.66  -456.86
...

```

Fig. 7: Sample STA report

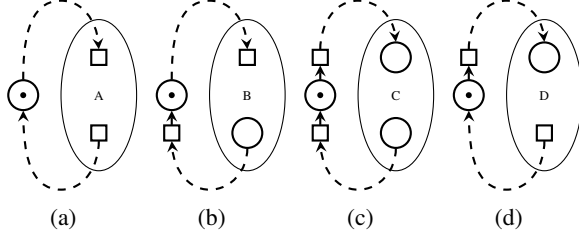


Fig. 8: Classification of token preserving regions (TPRs)

OpenSTA [20] as a base tool to interpret the cell characterization, to read and interpret the parasitics information and to compute combinational delays in the circuit. Fig. 7 shows a sample output of the tool. In Section IV we will describe the formulation of this delay tuning approach, its adequacy and feasibility. In Section VIII we will list out some possible improvements to automate this approach and ways to arrive at better initial approximations for the delays to reduce the number of iterations.

III. LOGIC SYNTHESIS FROM VIRTUAL CIRCUITS

In this section we describe the virtual circuit representation with construction rules for the control path Petri nets, data path node behavior and how we synthesize an asynchronous circuit as an isomorphic mapping to structural HDL code.

A. Construction rules of Petri nets used in AHIR

AHIR virtual circuits use *Type-2 Petri nets* to represent the control path, which are defined in [18]. We reproduce the same briefly for reference: A *Type-1 Petri net* is a live and

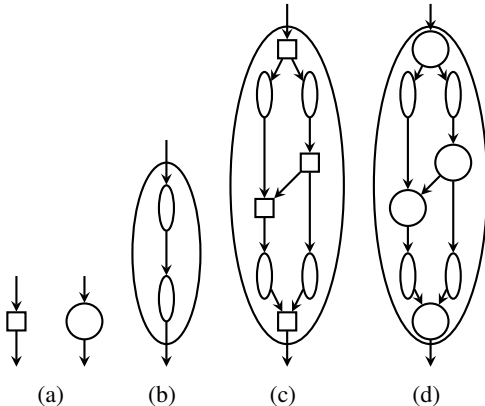


Fig. 9: Type-2 Petri net construction rules: (a) Primitive Regions (b) Series Region (c) Fork Region (d) Branch Region

1-safe Petri net that marks exactly one simple place in the initial marking. A *Token Preserving Region (TPR)* is a Petri net P that can be augmented with one simple place $\hat{p}$ and a sufficient number of simple transitions and edges, to produce a live and 1-safe Petri net P' such that $\hat{p}$ is the only marked place in the initial marking. A TPR has a unique *entry* place or transition and a unique *exit* place or transition. Depending on the type of entry or exit node, a TPR can be constructed in one of four ways called *classes* of TPR as shown in Fig. 8. The TPRs are unmarked 1-safe Petri nets. A Type-1 Petri net is constructed by placing a marked place around a TPR shown with dashed arcs in Fig. 8. A *Standard TPR (STPR)* is a TPR constructed using certain construction rules. These rules are shown in Fig. 9. Finally, a *Type-2 Petri net* is a Type-1 Petri net constructed using STPRs. Thus a Type-2 Petri net further restricts the Type-1 Petri nets by the construction rules of STPRs. [18] has shown that the Petri nets constructed using these rules are 1-safe and live.

B. Behavior of a data path node in AHIR virtual circuit

Some notation: 1) We denote the functionality of a data path operator as $D_{out} = f_{op}(D_{in})$, where D_{in} represents the input on the input data arc, f_{op} represents a function that the data path operator implements, D_{out} represents the output on the output data arc. 2) A *producer* is an entity that provides input data to a data path node. For the input ports of a circuit the *environment* acts as a producer. In all other cases another data path node acts as a producer. 3) A *consumer* is an entity that reads data from a data path node. For the output ports of a circuit the environment acts as a consumer. In all other cases another data path node acts as a consumer. 4) The control path interacts with a data path node using an interconnect protocol. To describe the protocol we use the symbolic notation described in Table III.

Each data path node in a virtual circuit has two interfaces: 1) A *write* or *sample* interface, consists of a request event called *sample request*, abbreviated *sreq* and a *sample ack* abbreviated *sack* and a data input D_{in} . We symbolically represent availability of a valid input D_{in} on the data input arc by event *din*. A data path node maintains a state in a register holding D_{out} , by computing $f_{op}(D_{in})$. We denote an undefined value of the data input as D_{in}^u and the event of its occurrence as *dinU*. To ensure validity of the state, following rules are imposed: a) A producer must ensure that the data path node receives valid data before raising a request to write it i.e. $din \circ sreq$. b) The data path node must issue an ack after completing the write operation. There must be exactly one ack issued after receiving a request. A producer must not raise a new request until an ack is received against an existing request i.e. $sreq \circ sack$ c) A producer must ensure that the data input remains valid until it receives an *sack* i.e. $din \circ sack$ d) Above property may be stated more explicitly as $[sreq, sack] \rightarrow \neg dinU$ 2) A *read* or *update* interface is similarly defined, which we omit for brevity. Fig. 10 shows both the interfaces as timing diagrams.

In a pipelined design, the communication is initiated by the producer. Hence, on system reset, the initial *ureq* event

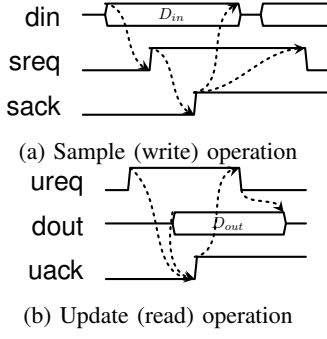


Fig. 10: Timing diagram of the data path interconnect protocol in AHIR virtual circuits

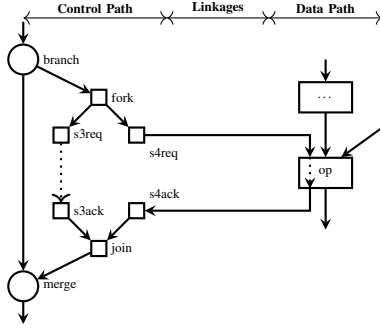


Fig. 11: An example of control path, data path and the linkages in AHIR

is implicitly triggered (described further in Section VI-C). Hence, any cyclic dependency of the form $ureq \odot E$, for some event E , flips to $E \odot ureq$. Also, the interval between $ureq$ and $uack$ where the data state must be valid becomes $[uack, ureq]$ since the first $ureq$ is raised after the first $uack$ in the pipelined mode. Thus:

Definition 1. 1) In the non-pipelined mode, the data path node's behavior for a given operator is defined by properties: a) Functional relation between the inputs and the outputs: $D_{out} = f_{op}(D_{in})$ b) Safety properties specifying the event intervals in which the data must be in a valid state: i) $[sreq, sack] \rightarrow \neg din \cup ii) [uack, \neg ureq] \rightarrow \neg dout \cup$ c) A set of cyclic dependencies that define the interconnect protocol: $\{din \odot sreq, sreq \odot sack, din \odot sack, dout \odot uack, ureq \odot dout, ureq \odot uack\}$ 2) Similarly, in the pipelined mode: a) $D_{out} = f_{op}(D_{in})$ b) Safety properties: i) $[sreq, sack] \rightarrow \neg din \cup ii) [uack, ureq] \rightarrow \neg dout \cup$ c) Cyclic dependencies: $\{din \odot sreq, sreq \odot sack, din \odot sack, dout \odot uack, dout \odot ureq, uack \odot ureq\}$

C. Synthesis as an isomorphic transformation of virtual circuit

Fig. 11 shows a unified view of control and data path in AHIR virtual circuits. It expands one of the dotted arcs in Fig. 2 to show the control - data path linkages. The ingredients of this view are: 1) A control path represented by a Type-2 Petri net. 2) A data path, a data flow graph where the nodes represent instances of operators and arcs represent the flow of

data between them. 3) Control - data linkages with two request - ack pairs, for write (sample) and read (update) operations.

Definition 2. AHIR virtual circuit is a graph $G_{vc} = (N_{vc}, E_{vc}, I_{vc})$ such that: 1) P is a set of unmarked places, T is a set of transitions, $F_c \subseteq (P \times T) \cup (T \times P)$ is a flow relation between places and transitions in the control path 2) D is a set of data path operator instances in the data path, $F_d \subseteq (D \times D)$ is a data flow relation between the instances, $F_{cd} \subseteq (T \times D)$: control - data interaction arcs that carry the request signals $F_{dc} \subseteq (D \times T)$: data - control interaction arcs that carry the ack signals $F_{br} \subseteq (D \times P)$: data path input to place nodes, used for conflict resolution 3) $T_n = \{\text{place}, \text{transition}, \text{data}\}$: a set of types that identifies the membership of the nodes to sets P, T, D respectively. 4) $T_e = \{\text{control}, \text{data}\}$ is a set of edge type. Edges from sets F_c, F_{cd} and F_{dc} are regarded as control edges and those from F_d and F_{br} are regarded as data edges. 5) $N_{vc} = P \cup T \cup D$, $E_{vc} = F_c \cup F_d \cup F_{cd} \cup F_{dc} \cup F_{br}$ 6) Each node $N \in N_{vc}$ is tuple (l, t_n) where l is unique label of N , $t_n \in T_n$ is type of N 7) Each edge $E \in E_{vc}$ that relates node N_s and N_t is tuple $(t_e, l_s, p_s, l_t, p_t)$ such that: $t_e \in T_e$ is type of E , $l_s = N_s.l$ and $l_t = N_t.l$ identify the source and target nodes by their labels respectively, $p_s, p_t \in \{0, 1, 2, \dots\}$ identify the ordinal position of the edge for a given edge type within the source and target node respectively. 8) I_{vc} is a set of request-ack pairs (t_{req}, t_{ack}) associated with an operator instance. Virtual circuit semantics require that there be a one safe and live Petri net dependency between t_{req} and t_{ack} , implemented by the operator. Each operator instance has two such request-ack pairs at ordinal positions 0 and 1 i.e. $I_{vc} = \{(req(d, i), ack(d, i)) \mid d \in D, i \in \{0, 1\}\}$ where $req(d, i) = ut$ ($t \in T, e \in F_{cd}, e.l_t = d.l, e.p_t = i, e.l_s = t.l$), $ack(d, i) = ut$ ($t \in T, e \in F_{dc}, e.l_s = d.l, e.p_s = i, e.l_t = t.l$)

The synthesis process generates code in structural form, which contains instantiation of hardware components from a predefined library of component types and interconnections between such component instances. Thus, structural code can be viewed as a directed graph, whose nodes are component instances and edges are the interconnections between their ports. We define the synthesized code as:

Definition 3. The synthesized code is graph $G_{ckt} = (N_{ckt}, E_{ckt}, I_{ckt})$, where the elements of the graph N_{ckt} , E_{ckt} and I_{ckt} follow the same annotation structure as G_{vc} in Definition 2. The role of each member of the annotation structure in the generated VHDL code is: 1) Each node $N \in N_{ckt}$ is tuple (l, t_n) where: l , the unique identifier of N , is used as the unique id of the generated component instance, $t_n \in T_n$ is passed as a compile time (generic) parameter with the instantiation of the component. 2) Each edge $E \in E_{ckt}$ is tuple $(t_e, l_s, p_s, l_t, p_t)$ where: $t_e \in T_e$, the arc type, identifies the component ports to connect, $l_s = N_s.l$ and $l_t = N_t.l$ identify the source and target component id respectively, $p_s, p_t \in \{0, 1, 2, \dots\}$ identify the ordinal position of the pin within a port to connect. 3) Implicit relation $I \in I_{ckt}$, has no explicit representation in the synthesized

code. The operator library implements this relation as defined as specified in Definition 2.

Definition 4. Synthesis function is $\mathcal{F}_{synth} : G_{vc} \mapsto G_{ckt}$, such that: $N_{ckt} = \{(N.l, N.t_n) \mid \forall N \in N_{vc}\}$, $E_{ckt} = \{(E.t_e, E.l_s, E.p_s, E.l_t, E.p_t) \mid \forall E \in E_{vc}\}$, $I_{ckt} = \{(I.t_{req}.l, I.t_{req}.t_n), (I.t_{ack}.l, I.t_{ack}.t_n) \mid \forall I \in I_{vc}\}$ i.e. there is an identity mapping between the nodes, edges and event labels from G_{vc} to G_{ckt} .

D. Mapping of Petri net to asynchronous circuit

Table I shows the gates we use to map the Petri net constructed using the construction rules described in Fig. 9.

We use the term *steady state* to mean either the initial state of a gate or its state after producing an output, till next change in any of its inputs. We use the term *usage cycle* to mean the time period when a steady state ends (with change in state of any of its inputs) and when a new steady state starts (by change in state of the output). In the context of a given usage cycle, we call the initial steady state as *precondition* and the next steady state as the *postcondition*.

In Table I, we use symbols t_0, t_1, t_2 to represent the event of transfer of a token over a Petri net arc. We describe the behavior of Petri net, in a usage cycle, in terms of these symbols as a Boolean formula and represent it as P . We use symbols a, b, c, d to represent wires in the digital circuit representation of the Petri net. Each Petri net arc maps to a wire in the digital circuit. We suffix the symbols that represent the wires with $_0$ to represent the precondition and $_1$ to represent the postcondition of a usage cycle. We describe the behavior of each gate as a Boolean formula in terms of these suffixed variables and represent it as G . We represent a Petri net's behavior in a usage cycle, at the input of a node by a Boolean formula I . Our mapping requires that in steady state a certain constraint over the state of its inputs holds. We represent this constraint as a Boolean formula E .

IV. FROM SYNTHESIZED CIRCUIT TO PHYSICAL DESIGN

For physical design we leverage off-the-shelf tools, except for the timing aspect. We formulate the timing problem for our approach and provide a solution to the same. Before that, we describe the problem of hazards in an asynchronous circuits.

A. Hazards in the controller circuit

In asynchronous circuits, a logic level transition on a control path wire represents an event. Hence any spurious transition may lead to malfunctioning of the circuit. Such spurious transition is called a *glitch* or a *hazard*. The problem is well studied in the literature [36], [47], [48]. We use the terminology used by [48] to introduce hazards. In Section V-C we will prove their absence in our synthesized circuits and state assumptions for the same.

B. Control - data event dependencies as timing constraints

The event dependencies such as $\text{din} \odot \text{sreq}$ (Definition 1) require realizing the data path events like din . In general,

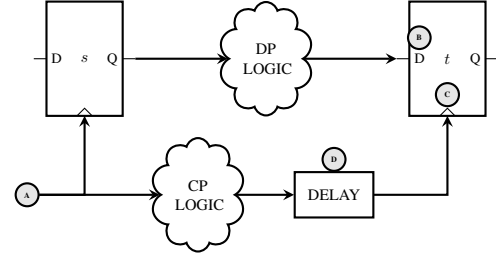


Fig. 12: Formulation of the timing problem

there are two ways to do this, *dual rail* and *bundled data*, which we discussed in Section I. Present work explores the latter, which involves ensuring that the control signal to a state holding element reaches its control input pin sufficiently after a valid data input has reached its data input pin. Fig. 12 shows a representative circuit that depicts this timing problem.

1) Let S be the set of all state holding elements in the circuit. 2) Let data input pin of a state holding element be represented by d and the control input pin by c . 3) Let $\cdot$ denote the relation between a component instance and its pin, e.g. $s.d$ represents pin d of component s . 4) Let $\bullet$ denote the driver of a pin i.e. $\bullet p$ for an input pin p denotes an output pin of some component that drives pin p . 5) Let $\rightarrow$ denote the data dependency between two state holding elements i.e. $s \rightarrow t$ denotes there is a combinational path from the data output pin of s to the data input pin of t . 6) Let $d_{min}(a, b)$ and $d_{max}(a, b)$ denote the minimum and maximum combinational path delay between an output pin a and an input pin b , respectively. 7) Let $d_{min}(D)$ represent the minimum combinational delay from the input to the output of a single-input single-output combinational component instance D . 8) Let D_n represent a delay element responsible for meeting the timing constraints for component instance n . 9) Let $t_{setup.max}(e)$ represent the maximum setup time of the state holding element e .

Thus, for the circuit in Fig. 12, s and t are state holding elements such that $s \rightarrow t$. The point marked B represents pin $t.b$ and C represents pin $t.c$. The point marked A represents the driver pin for $s.c$ i.e. $\bullet s.c$. Now, based on above discussion, the timing constraint that the circuit must meet is:

$$d_{max}(\bullet s.c, t.b) + t_{setup.max}(t) < d_{min}(\bullet s.c, t.c) \quad (1)$$

To meet this constraint, we have to insert a *delay element* D_t (marked D in Fig. 12), which has to meet constraint:

$$d_{min}(D_t) > d_{max}(\bullet s.c, t.b) + t_{setup.max}(t) - d_{min}(\bullet s.c, t.c) \quad (2)$$

In general, the data input of a state holding element may be influenced by more than one source elements. However, each state holding element has only one delay element to meet its timing constraints. Thus, each delay element may have multiple such constraints. Notice that there is only a lower bound on $d_{min}(D_t)$. Hence it is sufficient for D_t to satisfy the constraint that demands the maximum delay. Hence, the set of all delay constraints is given by:

$$\{d_{min}(D_t) > \max((d_{max}(\bullet s.c, t.b) + t_{setup.max}(t) - d_{min}(\bullet s.c, t.c)) \mid s \in S \mid s \rightarrow t) \mid t \in S\} \quad (3)$$

TABLE I: Mapping of Type-2 Petri nets to digital circuit and their behavioral equivalence

Petri net	Gate	Petri net behavior (P)	Input (I)	Map (M)	Gate behavior (G) (for i in 0,1)	Constraints (E)
Primitive nodes						
		$t_0 \Leftrightarrow t_1$	t_0	$t_0 \mapsto a$ $t_1 \mapsto b$	$a_i \Leftrightarrow b_i$	$a_0 \Leftrightarrow b_0$
Fork						
		$t_0 \Leftrightarrow t_1$ $t_0 \Leftrightarrow t_2$	t_0	$t_0 \mapsto a$ $t_1 \mapsto b$ $t_2 \mapsto c$	$a_i \Leftrightarrow b_i$ $a_i \Leftrightarrow c_i$	$a_0 \Leftrightarrow b_0$ $a_0 \Leftrightarrow c_0$
Join						
		$t_0 \wedge t_1 \Leftrightarrow t_2$	$t_0 \wedge t_1$	$t_0 \mapsto a$ $t_1 \mapsto b$ $t_2 \mapsto c$	$a_i \wedge b_i \Rightarrow c_i$ $\neg a_i \wedge \neg b_i \Rightarrow \neg c_i$ $(a_1 \oplus b_1) \Rightarrow (c_0 \Leftrightarrow c_1)$	$a_0 \Leftrightarrow b_0$ $a_0 \Leftrightarrow c_0$
Branch						
		$t_0 \wedge b_0 \Leftrightarrow t_1$ $t_0 \wedge \neg b_0 \Leftrightarrow t_2$	t_0	$t_0 \mapsto a$ $t_1 \mapsto c$ $t_2 \mapsto d$	$t_0 \wedge b_0 \Rightarrow t_1 \wedge \neg t_2$ $t_0 \wedge \neg b_0 \Rightarrow t_2 \wedge \neg t_1$ $\neg t_0 \Rightarrow \neg t_1 \wedge \neg t_2$	<i>true</i>
Merge						
		$t_0 \oplus t_1 \Leftrightarrow t_2$	$t_0 \oplus t_1$	$t_0 \mapsto a$ $t_1 \mapsto b$ $t_2 \mapsto c$	$a_i \oplus b_i \Leftrightarrow c_i$	$a_0 \oplus b_0 \Leftrightarrow c_0$

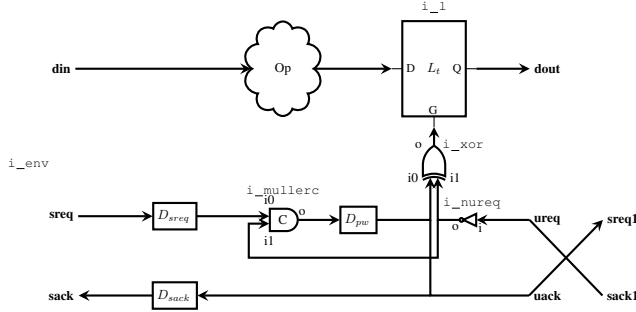


Fig. 13: A pipeline stage

Note that the system input and output ports can be trivially represented in the above formulation by modeling the environment as state holding elements.

C. Data path interconnect design and timing constraints

Fig. 13 shows our data path interconnect design, as a pipeline stage. Pins *din*, *dout* represent data input and outputs of the stage respectively. Pins *sreq*, *sack* form a request - ack pair for the *sample* (or write) operation, while pins *ureq*, *uack* form it for the *update* (or read) operation. L_t is the latch that holds the state, which is computed by *Op*, a combinational logic that implements some operator.

D_{sreq} corresponds to element marked *D* in Fig. 12, meant to meet the constraints in Eq. 3. D_{sack} delays the sample-ack signal to meet the hold time requirements of the latch. D_{pw} ensures that the Exor gate operates in fundamental mode, ensuring adequate gate pulse width. Thus, the hold time and

internal functioning of the stage require the following timing constraints:

$$d_{min}(i_mullerc.o, i_env.sack) > d_{max}(i_mullerc.o, i_l.G) + t_{hold.max}(L_t) \quad (4)$$

$$d_{min}(i_nureq.o, i_xor.i0) > d_{max}(i_nureq.o, i_xor.il) \quad (5)$$

Hence D_{sack} and D_{pw} have to meet the constraints:

$$d_{min}(D_{sack}) > d_{max}(i_mullerc.o, i_l.G) + t_{hold.max}(L_t) - d_{min}(i_mullerc.o, i_env.sack) \quad (6)$$

$$d_{min}(D_{pw}) > d_{max}(i_nureq.o, i_xor.il) - d_{min}(i_nureq.o, i_xor.i0) \quad (7)$$

Note that the delays in Eq. 6 and 7 are not a function of the combinational logic delay of the data path operator. Empirically, on most typical implementation setups, the RHS of these two equations is much smaller than the gate delay of a single buffer cell. Hence, typically, introduction of a single delay buffer in place of D_{sack} and D_{pw} meets these two constraints.

Pipelined and non-pipelined usage of the interconnect: As described with Definition 1, in a pipelined mode the circuit raises the *ureq* request implicitly on initialization. The circuit in Fig. 13 uses inverter *i_nureq* to raise the request implicitly. In a non-pipelined usage, the initial *ureq* request has to be raised explicitly by a consumer. In such usage, the interconnect design replaces the inverter instance *i_nureq* with a wire.

TABLE II: Run time of STA algorithm for various experiments

GC	Regs	Delays	Deps	Runtime (s)
1969	469	52	394	2.85
3766	956	92	800	12.37
4057	253	58	896	12.59
4286	307	43	928	6.59
4411	361	40	960	7.26
5342	631	54	1120	10.22
6143	1412	246	728	4.73
15639	0	15	128	19.81

D. Static timing analysis tool to impose the timing constraints

Algorithm 1 describes our implementation that checks Eq. 3 holds.³ The algorithm takes min and max corners, the parasitics information and implementation netlist as inputs. We build an abstract view of the netlist where state holding elements (flip flops and latches) and the delay elements we insert for meeting the timing constraints as the Timing Nodes.

We visit each cell instance once (INITNODES) to build the abstract timing node graph, then each timing node once (BUILDCLKSRCSET) to identify the clock sources and each data dependency twice (in BUILDCLKDEPS and STA). Asymptotically, if N, E represent the node and data dependency count of the netlist, then the complexity of this algorithm is, $O(N + E)$. The complexity of the complete process, however, is a function of the OpenSTA API complexity as well. We provide empirical data of the experiments we did for circuits with various gate, delay, register and data dependency counts, for reference.

V. VERIFICATION

Now we establish that the mapping from virtual circuits to the synthesized circuit is isomorphic, followed by the behavioral equivalence between the Type-2 Petri nets and the synthesized controller. Then we establish data path node behavioral equivalence under the formulated constraints and show that these constraints are feasible, necessary and sufficient. Lastly we prove the trace equivalence between the virtual circuit and the synthesized circuit.

A. Isomorphism between virtual circuit and synthesized code

Theorem 1. *Virtual circuit graph G_{vc} and the synthesized graph $G_{ckt} = \mathcal{F}_{synth}(G_{vc})$ (Definitions 2–4) are isomorphic.*

In graph G_{vc} , nodes N_{vc} are identified by a unique label. Nodes are annotated with type information. The implicit edges I_{vc} are identified by pairs of nodes. The edges E_{vc} are identified as a relation between two nodes and annotated with type and ordinal position information. $\mathcal{F}_{synth}$ uses an identity function to map the node labels from G_{vc} to G_{ckt} and carries the additional structural information, such as node and arc types and arc ordinal positions to G_{ckt} . Since an identity function is invertible, each element in G_{ckt} can be mapped back to its original element from G_{vc} , with the same type and ordinal position information. This shows that the function $\mathcal{F}_{synth}$ is a bijection. Hence the graphs G_{vc} and G_{ckt} are isomorphic. $\square$

³We omit the details of support routines to conserve space. The names of support routines should give an idea about their functionality.

Algorithm 1 STA algorithm

```

procedure BUILDCLKSRCSET( $REGS$ )
   $clksrcSet \leftarrow \emptyset$ 
  for each  $\{ r \in REGS \mid r \text{ is a data path reg} \}$  do
     $r.clksrc \leftarrow \text{FINDDRIVER}(r.Clk)$ 
    insert  $r.clksrc$  to  $clksrcSet$ 
     $r.clksrcClkMax \leftarrow$   $\leftarrow$ 
  GETMAXPATHDELAY( $r.clksrc, r.Clk$ )
     $r.clksrcQmax \leftarrow r.clksrcClkMax + r.tCQmax$ 
     $r.clksrcClkMin \leftarrow$   $\leftarrow$ 
  GETMINPATHDELAY( $r.clksrc, r.Clk$ )
     $r.clksrcQmin \leftarrow r.clksrcClkMin + r.tCQmin$ 
  end for
  return  $clksrcSet$ 
end procedure

procedure BUILDCLKDEPS( $clksrcSet$ )
  /*  $clkdepMap$  is a map of kind:
   $\{ clk \mapsto \{ clk \mapsto (DelayNode, C2CMinDelay) \} \}$  */
   $clkdepMap \leftarrow \emptyset$ 
  for each  $csrc \in clksrcSet$  do
     $cdeps \leftarrow \{ \text{nearest predecessor } p \mid p \in clksrcSet \}$ 
    for each  $cdep \in cdeps$  do
       $D \leftarrow \text{DelayNode on path cdep to csrc}$ 
      ASSERT( $D$  is unique)
       $Dmin \leftarrow \text{GETMINPATHDELAY}(cdep, csrc)$ 
      /*  $C2CMinDelay$  is by excluding  $D$ 's delay */
       $C2CMinDelay \leftarrow Dmin - D.Dmin$ 
       $clkdepMap[csrc][cdep] \leftarrow$   $\leftarrow$ 
      ( $D, C2CMinDelay$ )
    end for
  end for
  return  $clkdepMap$ 
end procedure

procedure STA
  INITSTA
  ( $REGS, DELAYS$ )  $\leftarrow$  INITNODES
   $clksrcSet \leftarrow \text{BUILDCLKSRCSET}(REGS)$ 
   $clkdepMap \leftarrow \text{BUILDCLKDEPS}(clksrcSet)$ 
  for each  $\{ t \in REGS \mid t \text{ is a data path reg} \}$  do
    for each  $s \in \text{FINDDRIVERS}(t.D)$  do
      /*  $(s, t)$  form a data dep pair */
       $AB \leftarrow \text{COMPUTEAB}(s, t)$ 
       $(d, AC) \leftarrow \text{COMPUTEAC}(s, t, clkdepMap)$ 
       $reqdDelay \leftarrow AB - AC$ 
      REQUIREDELAY( $d, reqdDelay$ )
    end for
  end for
  for each  $d \in DELAYS$  do
    PRINTDELAYREPORT( $d$ )
  end for
end procedure

```

B. Equivalence between Type-2 Petri nets and their mapping

Notion of equivalence between a Petri net and a digital circuit: We represent flow of token over a Petri net arc by a transition in the logic level over the wire that represents the arc. For example, in Petri net, when we say transition t_0 occurs, in the digital circuit we say the wire a representing its output arc, changes its logic level. Thus, the mapping $t_0 \mapsto a$ means, in a Boolean formula t_0 may be substituted with $a_0 \oplus a_1$. This substitution rule helps us formally assess the equivalence between a Petri net element and its corresponding digital circuit component.

On stability of the gate inputs:

Lemma 2. *In a circuit produced from Type-2 Petri nets, using mapping in Table I, for each gate, its input after an initial change causally depends on its output.*

From the construction rules of Type-2 Petri nets, observe that: 1) An STPR does not contain a marked place. A token has to enter the region via its unique entry point. Unless it exits the region via the unique exit point, it cannot re-enter the region. 2) Barring branch regions, other regions are acyclic. Hence, once a node receives a token, it can receive it again only after it leaves and re-enters the region. 3) A branch region permits cycles, but it disallows forks, which disallows replication of a token. And since there is no initial marking inside the region, there is at most one active token at any time in a branch region. 4) In the Petri net's mapping to a circuit, the flow of token is represented by a logic transition on a wire, which is responsible for change in input state of a gate. Due to above restrictions, once an input of a gate undergoes a state change, next change in it is causally dependent on its output to change. $\square$

A causal dependence is necessary but not sufficient to guarantee that the inputs to a gate are stable until its output completes a transition. We will revisit this when we analyze hazards in the circuit Section IV-A.

Outline of proof of equivalence between Type-2 Petri net and synthesized circuit: At first we restrict our treatment to at-most two input gates and generalize it in Lemma 6. Recall the notation introduced in Section III-D. 1) Basis the Type-2 Petri net construction rules, we prove the expressions I and E to be correct. 2) Using the notion of Petri net and digital circuit equivalence, we transform the Boolean formulas P and I in terms of the circuit variables, by using BDD composition, and call them P_{ckt} and I_{ckt} respectively. 3) Then we prove that the Petri net behavior P_{ckt} and the gate behavior G are equivalent if the constraint E holds i.e. $E \Rightarrow (P_{ckt} \Leftrightarrow G)$

Lemma 3. *Expression for input I described in Table I is valid.*

Given the Petri net is live by construction, except for merge node, all inputs of each node in the mapping table must receive a token in a usage cycle. Inputs of a merge place, on the other hand, have conflicting events on its inputs. Since the Petri net is 1-safe, in a given usage cycle, only one of the input transitions to a merge can occur. Hence Petri net behavior I in the table is valid. $\square$

We now compute P_{ckt} and I_{ckt} from P and I respectively, using the notion of equivalence and a BDD tool by composition as follows:

$$P_{ckt} = P[t \mapsto (w_0 \oplus w_1) \forall (t, w) \in M]$$

$$I_{ckt} = I[t \mapsto (w_0 \oplus w_1) \forall (t, w) \in M]$$

Lemma 4. *Steady state of Petri net nodes satisfies their respective constraint E in Table I.*

We prove this argument by induction, by taking initial state as a base case and induction step as the postcondition obtained by applying input I_{ckt} using characteristic function P_{ckt} of the Petri net node, under the inductive assumption that the precondition meets the constraint E and verifying that the postcondition, too, holds E .

Base case: In initial state of the circuit, the constraint E holds for each gate. Note that, primitive nodes, fork and merge are combinational circuits, whose constraint E is same as gate behavior G . Hence the constraint holds for these node types. The branch and join nodes are sequential circuits. We assume that the circuit imposes the given constraint by way of initializing the state of the storage elements to 0 on system reset. Thus for all node types, in initial state the constraint holds.

Inductive step: Given the Petri net node's characteristic function P_{ckt} and input I_{ckt} and under the inductive assumption that the constraint E holds in the precondition, the postcondition S_1 can be computed as: $S_1 = \exists(a_0, b_0, c_0, d_0)(P_{ckt} \wedge I_{ckt} \wedge E)$

To check if the constraint E holds in postcondition S_1 , we first express the constraint in terms of the postcondition variables i.e. $E_1 = E[a_0 \mapsto a_1 \dots d_0 \mapsto d_1]$

Now, if the postcondition meets the constraint, $S_1 \Rightarrow E_1$ must be true. We check that this expression to be a tautology using BDD tool for all node types, which proves that the inductive step holds and hence constraint E holds in all steady states of a node. $\square$

Theorem 5. *Gate mapping described in Table I faithfully represents respective element in the Type-2 Petri net.*

In Lemma 3 and 4 we established the correctness of I and E respectively. The Petri net behavior P follows from the Petri net semantics, which is equivalently expressed as P_{ckt} in terms of the circuit behavior and the gates are assumed to correctly implement the gate behavior G . Using BDD tool we verify that $E \Rightarrow (P_{ckt} \Leftrightarrow G)$ is a tautology, which establishes that a node is faithfully represented by the gate it maps to in Table I, under constraint E which is shown to hold in a Type-2 Petri net. $\square$

Generalizing the proof for multi-input gates:

Lemma 6. *Theorem 5 holds in presence of multi-input joins and merges.*

We can prove the identity $E \Rightarrow (P_{ckt} \Leftrightarrow G)$, by representing the expression P, G, I and E symbolically as input vectors of unknown size. We omit the proof for brevity.

C. Hazards in controller circuit

From mapping in Table I, the primitive nodes and forks map to wires and are trivially hazard free. The branch element receives an input from the data path for conflict resolution, involving a control - data interaction. We will be covering the treatment for such interaction in Section IV-B. We analyze the remaining two cases of joins and merges for potential hazards.

Function hazards :

Case 1: Exor function: Per Lemma 3, in one usage cycle, only one of the input transitions of a merge node can occur, which implies that only one of the inputs of the Exor gate can change i.e. the Exor gate operates in the *fundamental mode*. If the initial state is A and the changed state after the input transition is B , these states differ by exactly 1 bit. An Exor gate output changes when any one of its inputs changes i.e. $f(A) \neq f(B)$. Hence a static function hazard is absent. A dynamic function hazard requires a pair of intermediate states in the cube $[A, B]$. But since the states differ by only 1 bit, there are no intermediate states that can meet the criteria of the dynamic function hazard. Hence, the Exor function is free of function hazards in the context of the behavior of the merge in a Type-2 Petri net.

Case 2: Muller C function: Per Lemma 4, at the start of each usage cycle, the Muller C element inputs are in consensus. From the construction rules of a fork region in AHIR Type-2 Petri nets (Fig. 9) notice that, a token cannot re-enter a region unless it has left the exit point of the region, which is a join transition for a fork region. Hence it is impossible for a join to receive a new token, unless the join transition triggers. Hence, the state change in the inputs of Muller C element is monotonic in a given usage cycle.

Consider a usage cycle that starts with all inputs of Muller C element in state logic 0, say state A . The usage cycle ends when all inputs flip their state to 1 - say state B . The cube $[A, B]$ contains several intermediate states where one or more, but not all of the input states are 1. Let's call such set of states S_I . The set of feasible transitions can then be described as: 1) $\{A \rightarrow I \mid I \in S_I\}$ 2) $\{I_1 \rightarrow I_2 \mid I_1, I_2 \in S_I, I_1 \neq I_2\}$ 3) $\{I \rightarrow B \mid I \in S_I\}$ 4) $\{A \rightarrow B\}$

Note that an output state transition is possible only when the target state is B . Hence case 1 and 2 above need to be assessed for static function hazard and case 3 and 4 for dynamic function hazard. The only state that can cause an output transition is B , which is not a part of any transition cubes $[A, I]$ or $[I_1, I_2]$ in case 1 and 2. Hence static function hazard is not possible. Similarly, for case 3 and 4, no pair of states involving a state change is present in cubes $[I, B]$ or $[A, B]$ to cause a transition in the output. Hence a dynamic function hazard is not possible.

By symmetry, the above argument also applies when state A has all inputs at logic 1 and B has all inputs 0. Thus given the Type-2 Petri net construction rules and the initial state constraints, the Muller C element used as a join is free of function hazards.

Essential hazards :

In AHIR Type-2 Petri nets there are two ways to form a feedback path: 1) For the top TPR (Fig. 8) when a token leaves the TPR, it re-enters the TPR via a feedback loop. In

the circuit, this feedback loop consists of an inverter gate. 2) In a branch region cyclic paths are permitted. A token can re-enter a cyclic region after it exits the region, via a merge node.

Per Lemma 2, every gate's every input causally depends on the gate's output after an initial change. However, a gate output takes certain finite duration to complete a transition. The loads it drives may possibly infer the logic level it is transitioning to, before it has stabilized. To avoid essential hazard, we have to ensure that this change does not propagate to the same gate's inputs via a feedback path.

Formally, 1) Let $t_{rise.max}$ and $t_{fall.max}$ represent maximum time for an output to rise from logic level 0 to 1 and fall from 1 to 0, respectively. 2) Then maximum time to complete a transition, $t_{trans.max} = \max(t_{rise.max}, t_{fall.max})$ 3) Let G be set of all gates and I - a set of a gate's input pins, and o a gate's output pin. 4) Let $d_{min}(a, b)$ represent the minimum propagation delay from pin a to pin b . 5) Then the timing constraint required to be met is: $\forall g \in G, \forall i \in g.I : d_{min}(g.o, i) > t_{trans.max}$

As there is only a lower bound on the delay, this constraint is feasible and can be easily met by adding a delay element, with delay value $> t_{trans.max}$ to the path for the two scenarios mentioned above. Thus, the controller circuit is free of essential hazards, under the assumption that the feedback path around a TPR or via a merge node in a branch region has higher delay than the time required for a gate output to stabilize. On a pragmatic note, the feedback path in AHIR Petri nets is never a self loop i.e. at least one gate is involved in the feedback path, with path delay far exceeding $t_{trans.max}$. Hence an explicit insertion of delay to preempt essential hazards is unnecessary.

Logic hazards :

Logic hazards are a characteristic of an implementation of the function. The implementations may vary depending on the synthesis tool chain and underlying logic cell implementations. We state assumptions about the underlying cell implementations for the Exor and Muller C elements.

Case 1: Exor implementation : As shown before, the Exor gate in our controller operates in fundamental mode. Most of the implementation kits provide an Exor gate implementation that is hazard free in fundamental mode. It is typical for a cell library to include Exor gates up to 4 inputs. In situations where more inputs are involved, if the synthesizer produces a cascading of Exor gates, each gate in such cascaded arrangement operates in fundamental mode making the circuit free of logic hazards. However, it is imperative to examine the implementation to ascertain this. With the assumption that the underlying Exor implementation is free of logic hazards in fundamental mode, our merge implementation is free of logic hazards.

Case 2: Muller C implementation : Fig. 4 shows our implementation of a Muller C element. As we analyzed in the function hazard, the possible input transitions are restricted, with all inputs changing monotonically from logic 0 to logic 1 in a usage cycle and back to logic 0 in an alternate cycle. As described in the case of Exor implementation, we make an assumption that the implementations of the logic gates are free

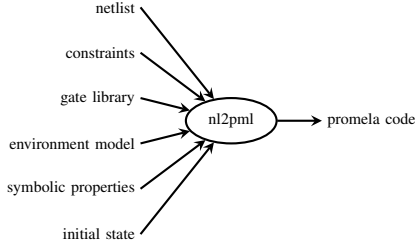


Fig. 14: Netlist2promela tool flow

TABLE III: Symbolic properties and their mapping to LTL

Symbolic property	Description	LTL formula
$\mathcal{L}(p)$	p changes infinitely often	$\neg((\Diamond \Box p) \vee (\Diamond \Box \neg p))$
$\mathcal{L}_{all}$	$\mathcal{L}(p) \forall p \in P$ where P is set of all pins	$\neg((\Diamond \Box p) \vee (\Diamond \Box \neg p)) \forall p \in P$
$a \odot b$	a and b strictly alternate starting with a . Initial state: $a = b = 0$	$\Box((a \rightarrow (a \mathcal{U} b))$ $\wedge (b \rightarrow (b \mathcal{U} \neg a))$ $\wedge (\neg a \rightarrow (\neg a \mathcal{U} \neg b))$ $\wedge (\neg b \rightarrow (\neg b \mathcal{U} a))$
$[a, b] \rightarrow c$	Property c holds in event interval $[a, b]$	$\Box((a \rightarrow (c \mathcal{U} b)) \wedge (a \rightarrow (\neg a \rightarrow (c \mathcal{U} \neg b))))$

of logic hazards. Given the set of possible transitions and this assumption, the gate and reset inputs to the latch are hazard free. Due to differing delays, a scenario of the gate acquiring logic 1 level while reset is yet to be logic 0, and the reverse of this can't be ruled out. However, in most latch implementations the reset gets precedence avoiding a race condition due to such scenario. Thus assuming the gates used to be free of logic hazards and precedence for the reset signal in the latch implementation, our Muller C implementation is free of logic hazards.

D. A Netlist to Promela converter tool

To assist in building further proofs using SPIN model checker, we have developed a tool netlist2promela [49] to convert a circuit netlist description to Promela, the modeling language of the SPIN model checker. Fig. 14 shows a context diagram of this tool.

The inputs to the tool are: 1) *netlist*: A custom json form to represent a circuit netlist. 2) *constraints*: A model checker explores all interleavings of unordered events in a circuit. But the delay elements in a circuit are meant to preempt some of the interleavings. Our tool supports modeling the behavior of the delays by expressing a constraint on occurrence of an event on a given pin, until a given condition is met. The condition is expressed in terms of the pin states and a special variable *val* that identifies the logic value that the event is about to set. Table IV, shows an example of how a delay element is modeled with a pin and a constraint. 3) *environment model*: The tool lets hand code additional processes in Promela, which is typically useful to model the environment, such as a producer and consumer component

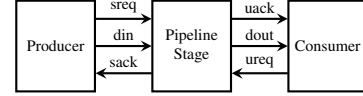


Fig. 15: Verification setup

TABLE IV: Constraints that model the delay elements

Delay Element	Pin	Constraint
D_{sreq}	$i_l.G$	$val \vee (i_env.din \oplus i_env.dout)$
D_{pw}	$i_xor.i0$	$(i_l.G \oplus (i_nureq.o \oplus i_xor.i0))$
D_{sack}	$i_env.sack$	$(i_l.G \oplus (i_nureq.o \oplus i_xor.i0))$ $\wedge (i_mullerc.o \oplus i_xor.i0)$

for the circuit under verification. 4) *gate library*: Behavior of various gates that are used in the circuit and that of wires, is coded in Promela in this library. 5) *symbolic properties*: The netlist2promela tool provides a symbolic notation for the properties one wishes to check. Table III describes the supported symbolic properties. The tool internally converts them into an LTL formulas. Further, our tool invokes another tool ltl2ba [50] to convert these properties into Promela code. 6) *initial state*: describes the initial logic level of all the pins. The state is modeled using the `byte` data type in Promela. 3-valued logic (a symbolic value of 2 to mean an undefined value D_{in}^u) can be used to define the initial state. The tool checks the consistency of these states with the gate behaviors and flags an error if the initial state is found to be inconsistent with the gate behaviors.

In the generated Promela code, each gate and wire is modeled as a concurrent process, which lets the model checker explore all possible interleavings, except those preempted by the constraints.

E. Data path interconnect equivalence

Verification setup: Fig. 15 describes our verification setup. We model the environment in the form of a *producer* and a *consumer* process. Both the producer and consumer run perpetually to help check that the properties of interest hold globally or infinitely often. For the non-pipelined mode verification, the consumer process initiates the first `ureq` request on initialization, while for the pipelined mode verification, the consumer waits for the `uack` signal on initialization. In both modes, the consumer flips the request `ureq` on receipt of a `uack`, to start a fresh cycle.

The producer inserts the undefined value D_{in}^u on the `din` between receiving of an `sack` and setting up of a new legitimate input on `din`, to help check the properties related the validity of the output. To help check the liveness, and to be able to detect the data readiness as an event, the producer flips the `din` value in each iteration. We model the operator to be an identity function. Initial state is modeled by setting all the control inputs to logic 0 and data inputs to D_{in}^u and setting the internal states consistently with these.

Table IV describes how we model the delay constraints: 1) D_{sreq} element's role is to enforce the timing constraint given by Eq. 1. The delay ensures that the `sreq` signal, which

is responsible for closing the latch gate, reaches the latch only after the data has reached and reflected on the latch output. The constraint applies only when closing the gate, which is represented by the logical or with *val*. 2) D_{pw} element's role is to ensure that both inputs of the Exor gate do not change simultaneously to ensure adequate gate pulse width. The input *i0* of the Exor gate is constrained to reflect this delay. 3) D_{sack} delays the *sack* until the Muller C element's output first reaches the latch gate.

Verification of the data path interconnect: With above setup, we verify the following results:

Lemma 7. *If the constraints in Table IV are satisfied and the environment (producer and consumer) follows the protocol Definition 1 for the respective usage modes, the pipeline stage design in Fig. 13 is deadlock free and implements the behavior in Definition 1.*

Using above verification setup, we verify the properties $\mathcal{L}_{all}$ and all the properties in Definition 1. $\square$

Lemma 8. *The set of constraints in Table IV is necessary and sufficient for the properties in Definition 1 to hold.*

Using the model checking setup described, above properties are verified, which shows the sufficiency of the constraints. Dropping any one constraint invalidates one or more of the properties, which shows the necessity of these constraints. $\square$

Lemma 9. *The constraints in Table IV are feasible.*

The delays D_{sack} and D_{pw} are internal to the pipeline stage as they control relative delays between two internal paths of a pipeline stage and have only a lower bound (Eq. 6, 7). The delay D_{sreq} is influenced by potentially multiple inputs that influence the combinational logic. As shown in Eq. 3 all constraints on this delay specify only a lower bound. Thus, all three constraints in Table IV are feasible. $\square$

Lemma 10. *The data path interconnect in Fig. 13 implements a 1-safe and live dependency between its requests and acks, as specified in Definition 2.*

The properties $sreq \odot sack$ and $ureq \odot uack$ are verified using above verification setup, which implies a strict alternation between the respective request and ack pairs. The property $\mathcal{L}_{all}$ shows that every request is eventually acknowledged. Hence, per request exactly one ack is produced. This is equivalent of a 1-safe and live dependency between the request and its respective ack. $\square$

Theorem 11. *Circuit in Fig. 13 faithfully represents a virtual circuit data path node for a given operator Op .*

1) We assume that the RTL library implements the operators correctly so that $D_{out} = f_{op}(D_{in})$ holds. 2) Lemmas 7 - 9 show that the design in Fig. 13 complies with the interconnect protocol (Definition 1), under the timing constraints in Table IV. Further, the constraints are shown to be feasible. 3) Lemma 10 shows that the design implements the implicit event dependencies that the virtual circuit specification (Definition 2). Hence the design faithfully represents a virtual circuit data path node for a given operator. $\square$

F. Trace equivalence between virtual and synthesized circuit

We show that any event trace that the synthesized circuit produces belongs to a set of feasible traces in its specification, for a given initial state and input. We prove this is using Prime Event Structures [40] representation, which model the causality and conflict relations in a system.

Definition 5. *Petri net is a tuple (B, E, F, M) where B is a set of conditions (places), E is a set of events (transitions), $F \subseteq (B \times E) \cup (E \times B)$ is flow or causal dependency relation (arcs) and $M \subseteq B$ is a marked subset of the conditions (places).*

A notion of *unfolding* or *occurrence net* of Petri nets was introduced by Ken McMillan [51]. Unfoldings are Petri nets that are acyclic and free of backward conflict (merge places), that represent causality and conflict relations in the original Petri net they are derived from.

Definition 6. *Given a 1-safe Petri net $P = (B_0, E_0, F_0, M_0)$ its occurrence net $P_{occ} = (B_1, E_1, F_1, M_1)$ is a 1-safe Petri net such that F_1 is acyclic and free of merges i.e. $\forall b \in B_1 | \bullet b | \leq 1$. There is a mapping λ from the places and transitions of P_{occ} to P . The flow relation F_1 is consistent with this mapping i.e. $\forall (a, b) \in F_1. (\lambda(a), \lambda(b)) \in F_0$. Initial marking M_1 is not preceded by any events in the relation F_1 i.e. $\forall b \in M_1. \bullet b = \emptyset$*

McMillan [51] introduced a notion of configuration. A configuration represents a feasible *partial run* of an occurrence net.

Definition 7. *A configuration C is a set of events of an occurrence net $P_{occ} = (B, E, F, M)$ such that $C \subseteq E$, C is causally closed i.e. $e \in C \Rightarrow e' \in C \forall e' \leq e$ where $e' \leq e$ represents a causal relation (path) from e' to e and C is conflict free i.e. $e_1, e_2 \in C \Rightarrow \neg(e_1 \# e_2)$. Relation $\#$ represents a conflict relation over events. When two events share a predecessor, they are in (direct) conflict i.e. $\bullet e_1 \cap \bullet e_2 \neq \emptyset \Rightarrow e_1 \# e_2$. The events causally dependent on conflicting events are in (inherited) conflict i.e. $e'_1 \# e'_2, e'_1 \leq e_1, e'_2 \leq e_2 \Rightarrow e_1 \# e_2$*

Let PN_{vc} represent a Petri net view of the graph G_{vc} (Definition 2) and PN_{ckt} represent the control path view of the synthesized graph G_{ckt} (Definition 3), both with explicit representation for the request-ack dependencies via the data path operators (Lemma 10). Let $\mathcal{M} : PN_{vc} \rightarrow PN_{ckt}$ represent the mapping between the representations. As we showed in Theorem 1 $\mathcal{M}$ is a bijection.

We prove the trace equivalence in two steps. First, we prove that the feasible configurations producible by either representation are in bijection. Second, we show that any event trace in the synthesized circuit is a subset of feasible traces in the virtual circuit, by linear expansion of configurations into traces.

Lemma 12. *Mapping $\mathcal{M}$ is causality preserving i.e. $n_1, n_2 \in PN_{vc}, n_1 \leq n_2 \Rightarrow n'_1 \leq n'_2$ where $n'_1 = \mathcal{M}(n_1), n'_2 = \mathcal{M}(n_2)$.*

In a Petri net, except for a merge place, the arcs form a causal relation for a node. Per mapping in Table I, primitive nodes, forks and branches map to gates with a single input and

their behavior G shows that their output is causally dependent on the input. The joins map to Muller C element and given the constraint E , all incoming wires must undergo a transition for the output transition to trigger. Thus, except for the gate that the merge place maps to all wires represent a causal relation. A configuration is based on an unfolded occurrence net of the respective representation in which the merge node draws its inputs from one of its predecessors on which it is causally dependent. Thus the mapping $\mathcal{M}$ is causality preserving. $\square$

Lemma 13. Mapping $\mathcal{M}$ is conflict preserving i.e. $n_1, n_2 \in PN_{vc}, n_1 \# n_2 \Rightarrow n'_1 \# n'_2$ where $n'_1 = \mathcal{M}(n_1), n'_2 = \mathcal{M}(n_2)$.

We introduced the terms *direct conflict* and *inherited conflict* in Definition 7. As per the Petri net semantics, when a place node has more than one successors, they are in a direct conflict. Such branch place maps to a branch element. The behavior of the branch element (G in Table I) shows that the successors of branch element are in conflict as only one of its output wires undergo a logic level transition, per configuration. Thus, the direct conflict relation is preserved in the mapping. Since the inherited conflict is a function of the causal relation, which is also preserved (Lemma 12), the mapping preserves the conflict relation. $\square$

Lemma 14. For any configuration C_{vc} produced in PN_{vc} , there exists a configuration C_{ckt} in PN_{ckt} such that C_{vc} and C_{ckt} are in bijection.

We prove this argument by inductively constructing C_{vc} using Definition 7 and showing that for every construction step of C_{vc} , there exists an equivalent step in C_{ckt} . Note that the configurations are sets, hence the sequence of triggering of events is not relevant.

Base case: Consider an empty configuration $C_{pre_{vc}} = \emptyset$ of PN_{vc} and $C_{pre_{ckt}} = \emptyset$ of PN_{ckt} . There exists a unique entry transition in a Type-2 Petri net. Liveness implies that the entry transition can be triggered. Equivalent entry transition can be triggered in PN_{ckt} . The new configurations after addition of this event are $C_{post_{vc}} = \{en_{vc}\}$ and $C_{post_{ckt}} = \{en_{ckt}\}$. By symmetry of the argument, $C_{post_{vc}}$ and $C_{post_{ckt}}$ are in bijection.

Inductive step: Given configurations $C_{pre_{vc}}$ and $C_{pre_{ckt}}$ which are in bijection (by inductive assumption), consider an event e_{vc} that can be added to $C_{pre_{vc}}$. Hence e_{vc} must meet two criteria in Definition 7: 1) Causally closed: i.e. $e' \leq e_{vc} \Rightarrow e' \in C_{pre_{vc}}$ 2) Conflict free: i.e. $e' \in C_{pre_{vc}} \Rightarrow \neg(e' \# e)$. But since, the causality and conflict relations are preserved through bijective mapping $\mathcal{M}$ (Lemma 12 and 13) an equivalent event e_{ckt} must exist, that can be added to $C_{pre_{ckt}}$. After these additions, $C_{post_{vc}} = C_{pre_{vc}} \cup \{e_{vc}\}$ and $C_{post_{ckt}} = C_{pre_{ckt}} \cup \{e_{ckt}\}$. $C_{pre_{vc}}$ and $C_{pre_{ckt}}$ are in bijection and there exists a mapping between e_{vc} and e_{ckt} . By symmetry of the construction process, the inverse mapping exists. Hence $C_{post_{vc}}$ and $C_{post_{ckt}}$ are in bijection. From the construction rules PN_{vc} has a unique exit node and from the liveness of the Type-2 Petri nets, the exit node is reachable. Hence the configuration construction process terminates and a configuration is finite. By principle of structural induction, configurations of PN_{vc} and PN_{ckt} are in bijection. $\square$

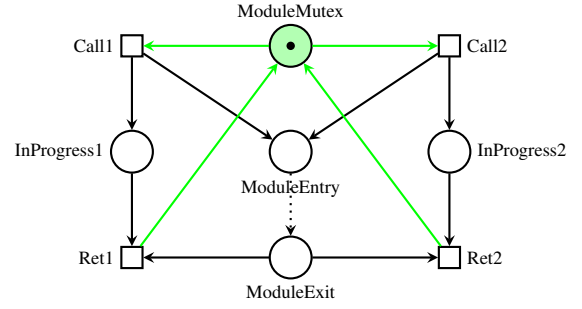


Fig. 16: Petri net model of an arbiter

Given a configuration, which is a causally closed and conflict free set of events, a set of feasible traces can be constructed by linear extension (also called linearization or topological sort). We define it as a linearization operator as follows:

Definition 8. For any configuration C , built using causal relation $\leq$, the set of feasible traces is given by linear expansion denoted as $Lin(C, \leq)$, such that $\forall T \in Lin(C, \leq)$, T is a total order $\leq^*$ such that $\forall x, y \in T, x \leq y \Rightarrow x \leq^* y$.

Theorem 15. An event trace produced by our synthesized circuit belongs to the set of feasible traces of the AHIR virtual circuit.

Let C_{ckt} be any configuration producible by the controller circuit PN_{ckt} . The set of feasible traces for this circuit is then $Lin(C_{ckt})$. For any event trace T_{ckt} produced by the circuit $T_{ckt} \in Lin(C_{ckt})$. As per Lemma 14, for the given configuration C_{ckt} , there must exist an equivalent configuration C_{vc} for PN_{vc} , with both configurations being in bijection. The set of feasible traces for this configuration is $Lin(C_{vc})$. Since the causal relation is preserved by mapping $\mathcal{M}$ (Lemma 12), $Lin(C_{ckt}) = Lin(C_{vc})$. (Recall the identity mapping in labels of nodes, used in Theorem 1). $T_{ckt} \in Lin(C_{ckt}), Lin(C_{ckt}) = Lin(C_{vc}) \Rightarrow T_{ckt} \in Lin(C_{vc})$ Hence any event trace produced by the synthesized circuit belongs to the set of feasible traces of the virtual circuit. $\square$

VI. ADVANCED TOPICS

Now we show how we model some of the advanced features of AHIR framework, using a Petri net pattern, we call *Passive Branch place* and its implementation. A passive branch is a Petri net place with more than one successors, such that the successors have other non-passive branch predecessor(s) that partly enable the transitions to fire. The passive branch, resolves the conflict by arbitrarily choosing the eligible successor to fire. A *Mutex Place* is just a special case of Passive Branch with initial marking of 1.

A. Arbiters for shared module invocation

Fig. 16 shows a Petri net model of a call arbiter that manages exclusive access to a module, depicted in green. The Module Exit place is a passive branch, which passes the token to the return transition of the caller that was served.

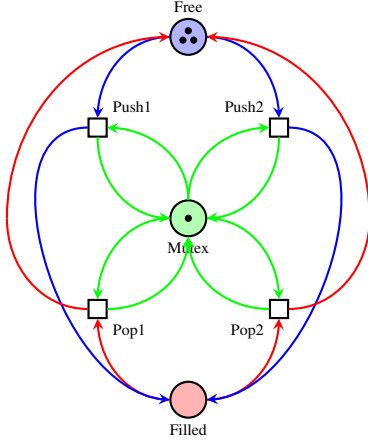


Fig. 17: Petri net model of communication pipes in AHIR

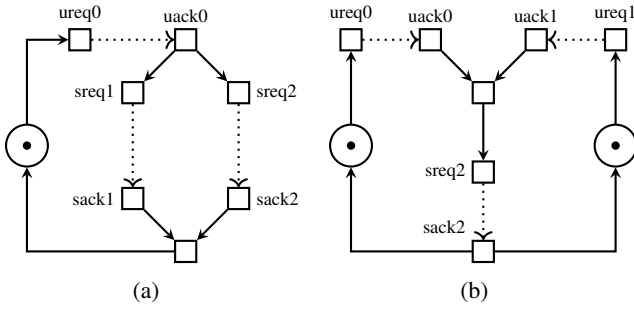


Fig. 18: Petri net model of pipelined interconnects: (a) one producer two consumers (b) two producers one consumer

B. Communication channels or pipes

AHIR framework supports communication channels or pipes [42]. A pipe is associated with a *depth*, which decides the number of buffers available in a pipe. Fig. 17 shows how we model the pipe controller. The part highlighted in green shows exclusiveness of pipe access using a Mutex place. The place named Free, is a passive branch, with an initial marking that represents the pipe depth. The blue arcs show the flow for a push operation that moves a token from the Free place to a passive branch place named Filled. The red arcs show the opposite flow for the pop operations.

C. Pipelined loops

AHIR framework has support for generating a pipelined design from an imperative do-while loop, in which each data path operation represents a pipeline stage. The implementation of this involves the following aspects:

Pipelined interconnect between data path elements: The pipelined interconnect in AHIR is covered in [52]. It is based on the interconnect protocol we described in 1. Fig. 18 shows how AHIR virtual circuit models the interconnect for two representative scenarios. The dotted arcs show implicit dependencies that the data path operator implements. A marked place ensures that the first update (read) request on system initialization is implicitly enabled.

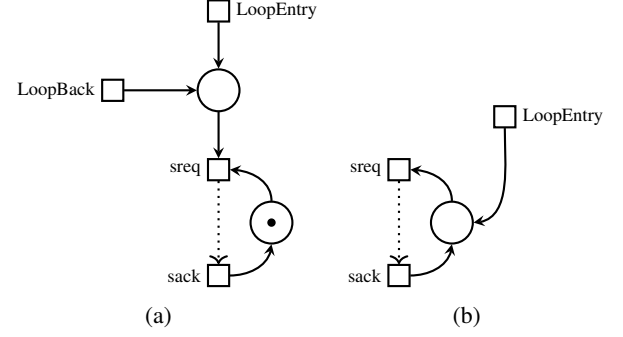


Fig. 19: (a) Petri net model of loop entry in AHIR virtual circuit and (b) its peephole optimization for non terminating loops

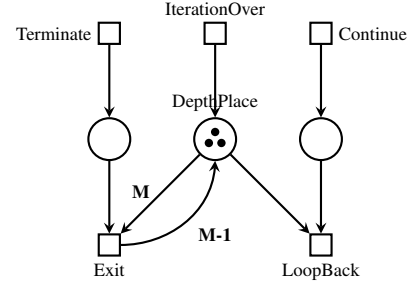


Fig. 20: Petri net model of pipelined loop termination in AHIR

Pipelined loop entry: Fig. 19a shows the Petri net model of entering a pipelined region in AHIR virtual circuits. The LoopEntry transition represents the first time entry into the loop, while LoopBack represents an iterative entry. sreq represents the sample request of the data path operator that depends on the loop entry.

Pipelined loop exit controller: In a pipelined loop, a new iteration of a loop can begin as soon as the loop continuation condition resolves to true, while honoring the data path dependencies. Multiple iterations of a loop may be concurrently active, with different data path elements potentially operating in the context of different iterations. When the loop continuation condition becomes false, the exit controller has to wait till all active iterations are over before exiting. This involves keeping a count of the iterations in-flight. AHIR specification language lets a designer specify *depth* M of a pipelined loop, which decides the number of bits to be used to implement a counter. We model this using Petri net, as shown in Fig. 20 using the Depth place as a passive branch and weighted arcs.

A peephole optimization for non-terminating pipelined loops: It is common for hardware pipelines to be perpetually running and in turn many pipelined loops may have the continuation condition to be always true. These loops do not need a loop exit controller described above. We transform the loop entry model of Fig. 19a into that shown in Fig. 19b and eliminate the loop exit controller.

D. Passive Branch implementation

To implement passive branches, we introduce an additional arc by which a transition requests a token from a passive

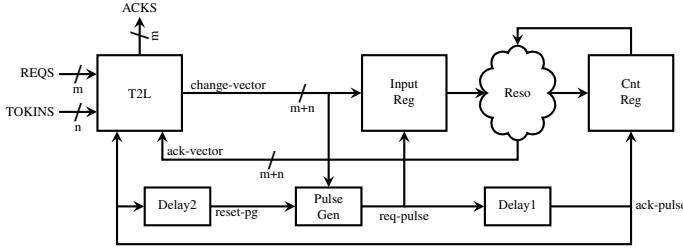


Fig. 21: Design of the passive branch element

branch. Fig. 21 shows a possible design of a passive branch place. The design supports m request arcs (REQS) and n incoming token arcs (TOKS) as inputs. There are m output arcs (ACKS) that correspond to the REQS. The T2L block is a transition-to-level converter that identifies the change-vector of the inputs. The pulse generator is triggered when there is a request to act upon and becomes insensitive to the input until it receives reset-pg pulse. Input Reg stores a stable snapshot of the input vector and Cnt Reg maintains a counter needed to honor the place capacity and arc weights. Delay1 accounts for the conflict resolution time taken by the combinational logic Reso and the meta stability filtering, while Delay2 holds back resetting of pulse generator until the change vector stabilizes.

We specialize the above design for some specific situations such as, when place capacity and arc weights are 1. In situations where the requests are guaranteed to be mutually exclusive, fundamental mode assumption is used to simplify the circuit.

VII. EXPERIMENTAL RESULTS

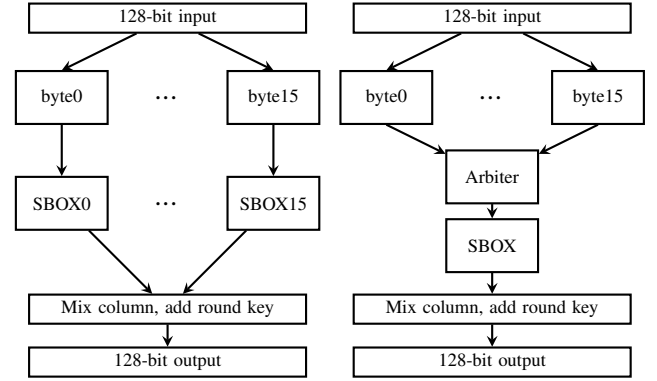
We now describe the experimental results of applying our methodology to the synthesis of an asynchronous AES algorithm [53], [54] implementation and compare them with some of the published results. Our encryption interface takes a 128-bit key and a 128-bit payload as an input and it produces a 128-bit cipher as an output.

We use pyaes [55] - a Python implementation of AES algorithm to generate the reference results for verification. We use a VHDL test bench for behavioral simulation of the generated RTL to verify the results. We use SPICE simulation to verify the implementation. We use a test bench written in C++⁴ and verify the SPICE simulation results with the reference results.

A. Design space exploration

128-bit AES encryption consists of 10 rounds of encryption. [57] shows a visualization of one round of the encryption flow. We club the Byte Sub and Shift Row steps into a Substitution Box (SBOX), implemented as a 128:1x8-bit multiplexer. Fig. 22 shows our encryption data flow which is implemented in Aa design entry language of the AHIR framework.

Table V describes the design space configurations that we explore and names each of them with a unique identifier for



(a) Throughput focus: parallel SBOXes (b) Area focus: shared SBOX and arbiter

Fig. 22: Our encryption round data flow

TABLE V: Description of the configurations explored.

Config	Objective	SBOXes	Arbiter	Pipelined
Our.16x	Throughput	16	-	-
Our.pl	Throughput	16	-	Each round as a stage
Our.16l	Area	1	16:1	-
Our.2l	Area	1	2:1	-
Our.4l	Area	1	4:1	-
Our.8l	Area	1	8:1	-

ease of reference. The configurations that fall under two different design objectives: *area-focused* and *throughput-focused*. The throughput focused configurations use 16 distinct SBOX instances, while the area focused ones use a single SBOX instance, shared using an arbiter.

AHIR framework supports describing pipelined designs in the form of looping constructs in the Aa language. The configuration Our.pl uses each encryption round as a pipeline stage, while Our.16x is a non-pipelined, combinational design. Among the area focused configurations, we explore different cascaded arrangements of arbiters such as a single 16:1 arbiter instance and cascaded usage of 2:1, 4:1 and 8:1 arbiters, identified with respective names.

B. Simulation results

We restrict the SPICE simulation to 1 round of encryption for non-pipelined configurations and 2 rounds for the pipelined configuration. We extrapolate the results to 10 rounds of encryption. Computation of metrics we present, is extrapolated to 10 rounds, from SPICE the measurements. For the pipelined configuration, the SPICE measurements are based on a two-stage pipeline (two encryption rounds) running over two inputs. The gate count and area is extrapolated to ten-stage pipeline. The throughput is computed, basis the time duration between two outputs. For energy and power computations, the results are extrapolated assuming the pipeline is not starved for inputs.

Comparing results across the diversity of technology nodes is challenging. Stillmaker et al. articulate this well [74]. We use their DeepScale tool [75], to normalize the time, power and area to 130 nm node used in our experimentation. Fig. 23 shows sorted scatter plots of normalized FoMs. We exclude the

⁴We have created a general purpose component [56] to help write NGSPICE test benches as reactive system that interacts with a circuit under test

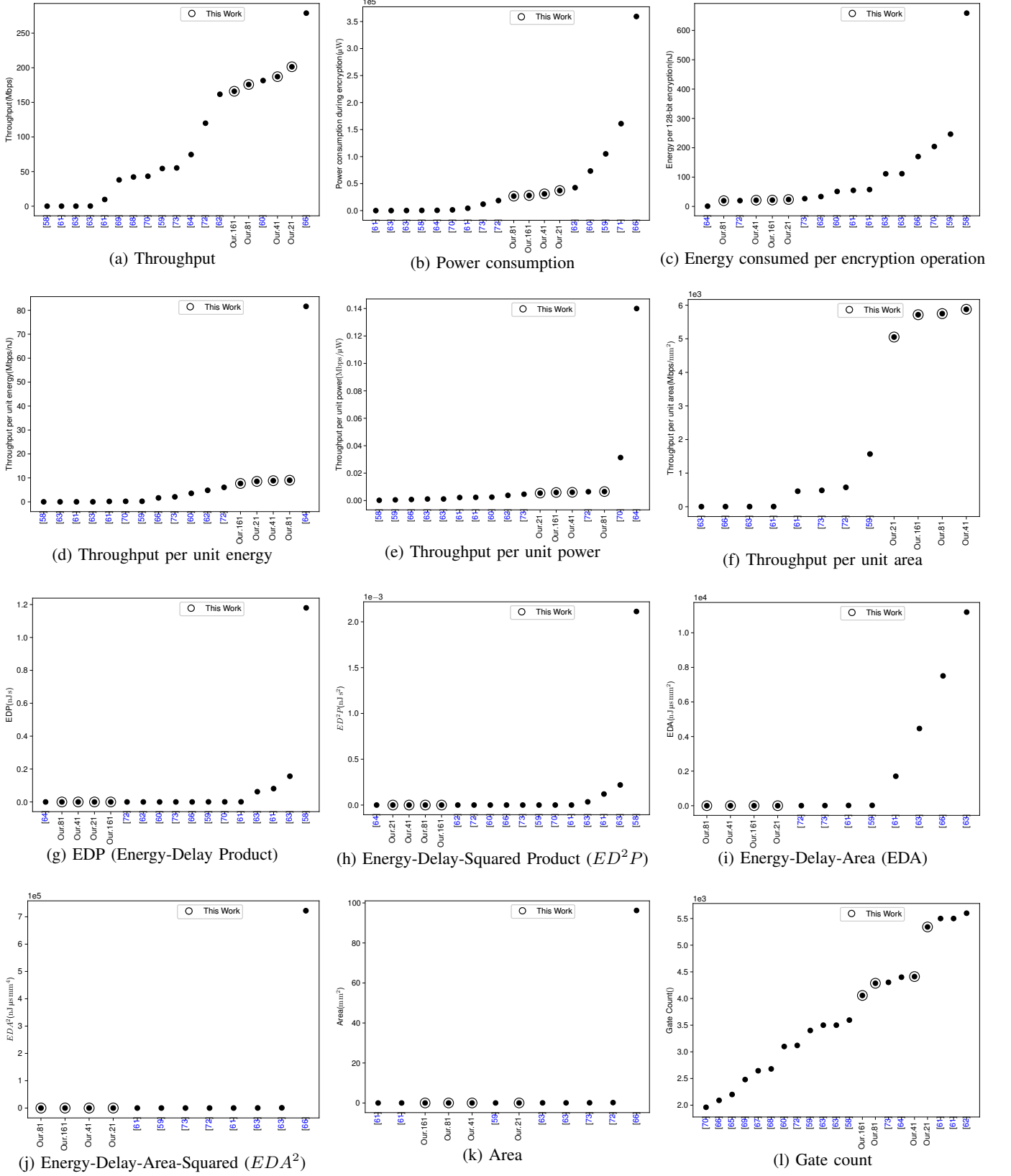


Fig. 23: Plots of normalized figures of merit for various AES implementations and our study, excluding our throughput focused configurations

TABLE VI: Various metrics gathered from surveyed AES implementations.

Ref	Year	Tech	Vdd	Thrpt	Power	Energy per op	Thrpt/ Energy	Thrpt/ Power	Thrpt/ Area	EDP	ED ² P	EDA	EDA ²	Area	GEs
		nm	V	Mbps	μ W	nJ	Mbps/ nJ	Mbps/ μ W	Mbps/ mm^2	nJ s	nJ s ²	nJ μ s mm^2	nJ μ s mm^4	mm^2	
[58]	2004	350	1.5	1.3e-2	1.2e+1	1.2e+2	1.0e-4	1.1e-3	-	1.2e+0	1.2e-2	-	-	-	3.6e+3
[59]	2005	350	1.5	9.9e+0	3.6e+3	4.6e+1	2.1e-1	2.8e-3	4.0e+1	6.0e-4	7.8e-9	1.5e+2	3.8e+1	2.5e-1	3.4e+3
[60]	2006	130	1.2	1.2e+2	5.6e+3	5.9e+0	2.1e+1	2.2e-2	-	6.2e-6	6.6e-12	-	-	-	3.1e+3
[61]	2010	130	0.75	3.6e-2	6.9e-1	2.5e+0	1.5e-2	5.2e-2	1.7e+0	8.8e-3	3.1e-5	1.8e+2	3.9e+0	2.1e-2	5.5e+3
[61]	2010	130	0.8	4.3e+0	9.9e+1	2.9e+0	1.5e+0	4.4e-2	2.1e+2	8.7e-5	2.6e-9	1.8e+0	3.8e-2	2.1e-2	5.5e+3
[62]	2010	180	1.8	1.0e+2	7.4e+3	9.1e+0	1.1e+1	1.4e-2	-	1.1e-5	1.4e-11	-	-	-	5.6e+3
[63]	2011	65	0.36	3.6e-2	2.5e-1	8.9e-1	4.0e-2	1.4e-1	2.0e+0	3.2e-3	1.1e-5	5.7e+1	1.0e+0	1.8e-2	3.5e+3
[63]	2011	65	0.4	1.0e-1	8.5e-1	1.1e+0	9.1e-2	1.2e-1	5.6e+0	1.4e-3	1.8e-6	2.5e+1	4.5e-1	1.8e-2	3.5e+3
[64]	2013	180	1.8	4.8e+1	9.3e+1	2.5e-1	1.9e+2	5.2e-1	-	6.6e-7	1.8e-12	-	-	-	4.4e+3
[65]	2013	180	-	4.0e+1	5.0e+3	3.5e+1	1.1e+0	8.1e-3	-	1.1e-4	3.6e-10	-	-	-	2.2e+3
[66]	2014	22	0.9	4.3e+2	1.3e+4	4.0e+0	1.1e+2	3.3e-2	1.6e+2	1.2e-6	3.5e-13	3.2e+0	8.8e+0	2.7e+0	2.1e+3
[67]	2016	90	-	9.4e+1	-	-	-	-	-	-	-	-	-	-	2.6e+3
[68]	2016	65	1.25	5.8e+1	-	-	-	-	-	-	-	-	-	-	2.7e+3
[69]	2017	65	1.25	5.2e+1	-	-	-	-	-	-	-	-	-	-	2.5e+3
[70]	2020	65	1.25	5.9e+1	1.8e+2	2.0e+1	3.0e+0	3.3e-1	-	4.2e-5	9.2e-11	-	-	-	2.0e+3
[71]	2020	180	1.5	-	1.6e+4	-	-	-	-	-	-	-	-	-	-
[72]	2021	130	0.8	5.3e+1	4.2e+2	1.0e+0	5.3e+1	1.3e-1	2.6e+2	2.4e-6	5.9e-12	5.1e-1	1.1e-1	2.1e-1	3.1e+3
[73]	2024	180	1.8	3.6e+1	2.1e+3	7.2e+0	4.9e+0	1.7e-2	1.6e+2	2.6e-5	9.4e-11	5.6e+0	1.2e+0	2.2e-1	4.3e+3
Our.16x	2025	130	1.8	2.6e+3	2.6e+4	1.3e+0	2.0e+3	9.9e-2	2.8e+4	6.4e-8	3.2e-15	6.0e-3	5.6e-4	9.3e-2	1.6e+4
Our.pl	2025	130	1.8	1.7e+4	2.4e+5	2.4e+0	6.8e+3	6.8e-2	1.6e+4	1.9e-8	1.5e-16	2.0e-2	2.1e-2	1.0e+0	1.7e+5
Our.16l	2025	130	1.8	1.7e+2	7.3e+3	5.6e+0	3.0e+1	2.3e-2	5.7e+3	4.3e-6	3.3e-12	1.3e-1	3.6e-3	2.9e-2	4.1e+3
Our.2l	2025	130	1.8	2.0e+2	9.6e+3	6.1e+0	3.3e+1	2.1e-2	5.1e+3	3.9e-6	2.5e-12	1.5e-1	6.2e-3	4.0e-2	5.3e+3
Our.4l	2025	130	1.8	1.9e+2	8.0e+3	5.5e+0	3.4e+1	2.3e-2	5.9e+3	3.8e-6	2.6e-12	1.2e-1	3.8e-3	3.2e-2	4.4e+3
Our.8l	2025	130	1.8	1.8e+2	6.9e+3	5.1e+0	3.5e+1	2.5e-2	5.7e+3	3.7e-6	2.7e-12	1.1e-1	3.4e-3	3.1e-2	4.3e+3

TABLE VII: Dynamic and idle power consumption

Config	Dynamic Power (μ W)	Idle Power (μ W)	Dy-to-idle Ratio
Our.16x	2.6e+4	1.4e+1	1.9e+3
Our.pl	2.4e+5	1.6e+2	1.5e+3
Our.16l	7.3e+3	3.2e+0	2.3e+3
Our.2l	9.6e+3	4.5e+0	2.1e+3
Our.4l	8.0e+3	3.8e+0	2.1e+3
Our.8l	6.9e+3	3.7e+0	1.9e+3

throughput focused configurations from the plots since their performance and resource metrics are in a different range than the area focused configurations. In Table VII we list the idle power consumption and the ratio of dynamic to idle power consumption for our experiments.

Some observations: 1) The results establish the feasibility of our asynchronous circuit synthesis methodology and show that the results are competitive. 2) The study also demonstrates the agility of our framework to explore the design space by creating and benchmarking various configurations with area-throughput trade offs. 3) Our throughput-focused configurations produce best in class throughput, although with higher area cost and power consumption. 4) Our area-focused configurations use a gate count and area that is comparable with other results, but produce better throughput than the most. 5) Both area and throughput focused configurations perform well in FoMs throughput-per-unit-area, energy-delay-area and energy-delay-area-square. 6) We observe that the idle power consumption of the asynchronous circuits produced is approximately 2000 times lower than the dynamic power consumption, which highlights a significant benefit of the asynchronous circuits.

VIII. FUTURE WORK

1) To enable our methodology to be a part of a larger VLSI ecosystem, such as chiplet based systems, which may use a mix of asynchronous and synchronous components, it will be useful to build protocol adapters to interoperate with synchronous components. Interfacing with external memory (RAM) is often going to be a requirement in many chip designs. 2) Post-implementation tuning of time delays, to meet the timing constraints, is a manual step involved in our flow. Our delay elements are designed with a *fixed* and a *variable* delay component, such that the variable component can be changed post-implementation, using the constant bits embedded in the design. However, currently we do not have an estimation model for the delay that can take benefit of this ability. The delay required is a function of the operator(s) that constitute the combinational logic of a pipeline stage, the data width, the placement and routing and the technology node. It is possible to empirically arrive at a prediction model that predicts the delay requirements as a range of upper and lower bounds. The lower bound can be used to specify the fixed component of the delay and the slack between the lower and upper bound to specify the variable component. This can speed up the process of delay tuning and potentially automate it completely. 3) If we can make placement and routing aware of the control and data path distinction and can give higher priority to data path nets, it will reduce the delay requirements, leading to area and energy saving as well as potentially reduction in the latency. 4) Off-the-shelf cell libraries often do contain delay elements that make use of buffers with larger input areas to produce delays. However, a bundled data design we often need to cascade multiple such elements, to meet the delay constraints. Besides increased area, these also

increase the energy consumption by the delay elements. The physical design aspect of these elements can be explored to create delay elements with suitable characteristics that meet the delay requirements with fewer stages, thus reducing the area and energy costs. 5) Our experimental results are based on typical operating conditions of supply voltage and temperature. Further exploration may be taken up with greater variation in these parameters to assess their impact on various figures of merit. We may also explore more classes of applications to assess the suitability of the methodology and asynchronous paradigm.

IX. CONCLUSION

1) We demonstrated a high-level synthesis framework for asynchronous circuits with a flow from imperative programs to RTL that can feed to any off-the-shelf ASIC flow built for the synchronous design. Our framework contains a static timing analysis tool to tune the delays to meet the timing constraints, which we showed to be feasible, necessary and sufficient. We proposed some ways to minimize or potentially eliminate the manual intervention for the timing closure. 2) We formalized the problem of synthesis of asynchronous circuits using a restricted subset of Petri nets that AHIR framework builds. Compared to the established methods like STGs, our Petri net formulation is simpler because we can isomorphically map it to a synthesized circuit, without involving complete state coding problem like STGs. 3) Using our methodology, we designed a circuit for AES encryption. We explored the design space, with different area-latency-energy trade offs. We performed transistor level simulation using SPICE and compared the metrics gathered with various published results. We find our results to be competitive; particularly the energy efficiency and idle power consumption show that such methodology could fit the use cases where these metrics matter.

REFERENCES

- [1] V. Mehrotra and D. Boning, "Technology scaling impact of variation on clock skew and interconnect delay," in *Proceedings of the IEEE 2001 International Interconnect Technology Conference (Cat. No.01EX461)*, 2001, pp. 122–124.
- [2] H. Homayoun, S. Golshan, E. Bozorgzadeh, A. Veidenbaum, and F. J. Kurdahi, "On leakage power optimization in clock tree networks for asics and general-purpose processors," *Sustainable Computing: Informatics and Systems*, vol. 1, no. 1, pp. 75–87, 2011. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2210537910000065>
- [3] D. Duarte, V. Narayanan, and M. Irwin, "Impact of technology scaling in the clock system power," in *Proceedings IEEE Computer Society Annual Symposium on VLSI. New Paradigms for VLSI Systems Design. ISVLSI 2002*, 2002, pp. 59–64.
- [4] K. Agarwal, H. Deogun, D. Sylvester, and K. Nowka, "Power gating with multiple sleep modes," in *7th International Symposium on Quality Electronic Design (ISQED'06)*, 2006, pp. 5 pp–637.
- [5] L. A. Barroso and U. Hözlze, "The case for energy-proportional computing," *Computer*, vol. 40, no. 12, pp. 33–37, 2007.
- [6] R. Buyya, S. Ilager, and P. Arroba, "Energy-efficiency and sustainability in new generation cloud computing: A vision and directions for integrated management of data centre resources and workloads," 2023. [Online]. Available: <https://arxiv.org/abs/2303.10572>
- [7] R. A. de Cristo, R. P. Jasinski, and V. A. Pedroni, "Analysis and preliminary measurements of radiated emissions in an asynchronous circuit versus its synchronous counterpart," in *2010 International Conference on Reconfigurable Computing and FPGAs*, 2010, pp. 127–131.
- [8] Z. Yu, S. Furber, and L. Plana, "An investigation into the security of self-timed circuits," in *Proceedings - International Symposium on Asynchronous Circuits and Systems—Proc. Int. Symp. Asynchr. Circuits Syst.* United States: IEEE Computer Society, 2003, pp. 206–215, 9th IEEE International Symposium on Asynchronous Circuits and Systems, ASYNC 2003 ; Conference date: 01-07-2003. [Online]. Available: <http://ieeexplore.ieee.org/iel5/8538/26997/01199173.pdf?tp=&arnumber=1199173&isnumber=26997>
- [9] B. Ghavami, H. Pedram, and M. Najibi, "An eda tool for implementation of low power and secure crypto-chips," *Computers & Electrical Engineering*, vol. 35, no. 2, pp. 244–257, 2009, circuits and Systems for Real-Time Security and Copyright Protection of Multimedia. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0045790608000566>
- [10] G. Bouesse, M. Renaudin, S. Dumont, and F. Germain, "Dpa on quasi delay insensitive asynchronous circuits: Formalization and improvement," in *Design, Automation and Test in Europe*. IEEE, p. 424–429. [Online]. Available: <http://dx.doi.org/10.1109/DATe.2005.124>
- [11] W. Cilio, M. Linder, C. Porter, J. Di, D. R. Thompson, and S. C. Smith, "Mitigating power- and timing-based side-channel attacks using dual-spacer dual-rail delay-insensitive asynchronous logic," *Microelectronics Journal*, vol. 44, no. 3, pp. 258–269, 2013. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0026269212002418>
- [12] A. B. Amar, A. B. Kouki, and H. Cao, "Power approaches for implantable medical devices," *Sensors*, vol. 15, no. 11, pp. 28 889–28 914, 2015. [Online]. Available: <https://www.mdpi.com/1424-8220/15/11/28889>
- [13] M. J. Laskar, A. Banerjee, V. Prasad, and D. Das, "Cmos circuits in the design and development of implantable medical devices," *Biomedical Materials & Devices*, vol. 4, no. 1, pp. 132–142, Mar 2026. [Online]. Available: <https://doi.org/10.1007/s44174-024-00267-0>
- [14] A. B. Rapeaux and T. G. Constandinou, "Implantable brain machine interfaces: first-in-human studies, technology challenges and trends," *Current Opinion in Biotechnology*, vol. 72, pp. 102–111, 2021, tissue, Cell and Pathway Engineering. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S095816692100183X>
- [15] M. Yip, P. Bowers, V. Noel, A. Chandrakasan, and K. M. Stankovic, "Energy-efficient waveform for electrical stimulation of the cochlear nerve," *Scientific Reports*, vol. 7, no. 1, Oct. 2017. [Online]. Available: <http://dx.doi.org/10.1038/s41598-017-13671-y>
- [16] M. Prauzek, J. Konecny, M. Borova, K. Janosova, J. Hlavica, and P. Musilek, "Energy harvesting sources, storage devices and system topologies for environmental wireless sensor networks: A review," *Sensors*, vol. 18, no. 8, 2018. [Online]. Available: <https://www.mdpi.com/1424-8220/18/8/2446>
- [17] P. Purohit and R. Manohar, "Field-programmable encoding for address-event representation," *Frontiers in Neuroscience*, vol. 16, Dec. 2022. [Online]. Available: <http://dx.doi.org/10.3389/fnins.2022.1018166>
- [18] S. D. Sahasrabudde, "A competitive pathway from high-level programs to hardware specifications," Ph.D. dissertation, IIT Bombay, 2009.
- [19] C. Petri, "Kommunikation mit Automaten," Ph.D. dissertation, Institut für instrumentelle Mathematik, Bonn, 1962.
- [20] J. Cherry and Contributors, "Opensta: Open-source static timing analysis," <https://github.com/The-OpenROAD-Project/OpenSTA>, 2020.
- [21] M. Shalan and T. Edwards, "Building openlane: A 130nm openroad-based tapeout- proven flow : Invited paper," in *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2020, pp. 1–6.
- [22] SkyWater Technology Foundry and Google LLC, "Skywater open source PDK," <https://skywater-pdk.readthedocs.io/>, 2020.
- [23] D. A. Huffman, "The synthesis of sequential switching circuits. journal of the franklin institute, vol. 257 (1954), pp. 161-190, 275-303," *Journal of Symbolic Logic*, vol. 20, no. 1, pp. 69–70, 1955.
- [24] S. Nowick and D. Dill, "Automatic synthesis of locally-clocked asynchronous state machines," in *1991 IEEE International Conference on Computer-Aided Design Digest of Technical Papers*, 1991, pp. 318–321.
- [25] I. E. Sutherland, "Micropipelines," *Commun. ACM*, vol. 32, no. 6, pp. 720–738, jun 1989. [Online]. Available: <https://doi.org/10.1145/63526.63532>
- [26] C. E. Molnar, "Synthesis of delay-insensitive modules," in *Chapel Hill Conference on VLSI Chapel Hill, NC, May 1985*, 1985.
- [27] J. T. Udding, "A formal model for defining and classifying delay-insensitive circuits and systems," *Distributed Computing*, vol. 1, no. 4, pp. 197–204, Dec. 1986. [Online]. Available: <http://dx.doi.org/10.1007/BF01660032>
- [28] A. J. Martin, *The Limitations to Delay-Insensitivity in Asynchronous Circuits*. New York, NY: Springer New York, 1990, pp. 302–311. [Online]. Available: https://doi.org/10.1007/978-1-4612-4476-9_35

- [29] K. van Berkel, F. Huberts, and A. Peeters, "Stretching quasi delay insensitivity by means of extended isochronic forks," in *Proceedings Second Working Conference on Asynchronous Design Methodologies*, 1995, pp. 99–106.
- [30] R. Manohar and A. J. Martin, "Quasi-delay-insensitive circuits are turing-complete," USA, Tech. Rep., 1995.
- [31] R. Manohar and Y. Moses, "The eventual c-element theorem for delay-insensitive asynchronous circuits," in *2017 23rd IEEE International Symposium on Asynchronous Circuits and Systems (ASYNC)*, 2017, pp. 102–109.
- [32] R. Miller, *Switching Theory*. Krieger, 1979. [Online]. Available: <https://books.google.co.in/books?id=1e9MAQAAIAAJ>
- [33] T.-A. Chu, "On the models for designing vlsi asynchronous digital systems," *Integr. VLSI J.*, vol. 4, no. 2, pp. 99–113, jun 1986.
- [34] D. Chapiro, "Globally-asynchronous locally-synchronous systems," 09 1984.
- [35] P. A. Beerel, R. O. Ozdag, and M. Ferretti, *A Designer's Guide to Asynchronous VLSI*. Cambridge University Press, Feb. 2010. [Online]. Available: <http://dx.doi.org/10.1017/CBO9780511674730>
- [36] C. J. Myers, *Asynchronous Circuit Design*. Wiley, Jul. 2001. [Online]. Available: <http://dx.doi.org/10.1002/0471224146>
- [37] S. Hauck, "Asynchronous design methodologies: an overview," *Proceedings of the IEEE*, vol. 83, no. 1, pp. 69–93, 1995.
- [38] D. Edwards and A. Bardsley, "Balsa: An asynchronous hardware synthesis language," *The Computer Journal*, vol. 45, no. 1, pp. 12–18, 01 2002. [Online]. Available: <https://doi.org/10.1093/comjnl/45.1.12>
- [39] S. Ataei, W. Hua, Y. Yang, R. Manohar, Y.-S. Lu, J. He, S. Maleki, and K. Pingali, "An open-source eda flow for asynchronous logic," *IEEE Design & Test*, vol. 38, no. 2, pp. 27–37, 2021.
- [40] G. Winskel, "Event structures," in *Petri Nets: Applications and Relationships to Other Models of Concurrency*, W. Brauer, W. Reisig, and G. Rozenberg, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1987, pp. 325–392.
- [41] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently computing static single assignment form and the control dependence graph," *ACM Trans. Program. Lang. Syst.*, vol. 13, no. 4, pp. 451–490, oct 1991. [Online]. Available: <https://doi.org/10.1145/115372.115320>
- [42] M. Desai, "Aa language reference manual," https://github.com/madhavPdesai/ahir/blob/master/prebuilt_ubuntu_16.04_release/docs/pdf/AaLRM.pdf, 2024.
- [43] C. Lattner and V. Adve, "Llvm: a compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, 2004, pp. 75–86.
- [44] T. Gingold and Contributors, "Ghdl: An open-source vhdl simulator," <https://github.com/ghdl/ghdl>, 2003.
- [45] —, "Ghdl-yosys plugin: Vhdl synthesis with yosys using ghdl," <https://github.com/ghdl/ghdl-yosys-plugin>, 2020.
- [46] C. Wolf, "Yosys open synthesis suite," <https://yosyshq.net/yosys/>.
- [47] E. B. Eichelberger, "Hazard detection in combinational and sequential switching circuits," *IBM Journal of Research and Development*, vol. 9, no. 2, pp. 90–99, 1965.
- [48] S. M. Nowick, "Automatic synthesis of burst-mode asynchronous controllers," Ph.D. dissertation, Stanford University, Stanford, CA, USA, 1993.
- [49] "A converter from our custom netlist json format to promela for verifying properties," URL: <https://github.com/mayureshw/netlist2promela>.
- [50] P. Gastin and D. Oddoux, "Fast ltl to büchi automata translation," in *Computer Aided Verification*, G. Berry, H. Comon, and A. Finkel, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 53–65. [Online]. Available: https://doi.org/10.1007/3-540-44585-4_6
- [51] K. L. McMillan, "Using unfoldings to avoid the state explosion problem in the verification of asynchronous circuits," in *Computer Aided Verification*, G. von Bochmann and D. K. Probst, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 164–177.
- [52] M. Desai, "EE 789 – algorithmic design of digital systems," Graduate course lectures, Department of Electrical Engineering, IIT Bombay, 2021, unpublished course material.
- [53] J. Daemen and V. Rijmen, *The design of Rijndael*. Springer, 2002, vol. 2.
- [54] National Institute of Standards and Technology (NIST), "Advanced Encryption Standard (AES): FIPS PUB 197 (Updated)," U.S. Department of Commerce, FIPS PUB 197-upd1, Sep. 2023, available online: <https://doi.org/10.6028/NIST.FIPS.197-upd1>. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.197-upd1>
- [55] R. Moore, "pyaes: Pure-python implementation of the aes block-cipher," <https://github.com/ricmoo/pyaes>, 2019, version 1.6.1. [Online]. Available: <https://github.com/ricmoo/pyaes>
- [56] M. Warunjikar, "spicetools: C++ interface to ngspice library for digital circuit simulation," <https://github.com/mayureshw/spicetools>, 2024.
- [57] Wikipedia contributors, "Advanced encryption standard," https://en.wikipedia.org/wiki/Advanced_Encryption_Standard, 2026, accessed: 2026-05-21.
- [58] M. Feldhofer, S. Dominikus, and J. Wolkerstorfer, "Strong authentication for rfid systems using the aes algorithm," in *Cryptographic Hardware and Embedded Systems-CHES 2004: 6th International Workshop Cambridge, MA, USA, August 11-13, 2004. Proceedings 6*. Springer, 2004.
- [59] M. Feldhofer, J. Wolkerstorfer, and V. Rijmen, "Aes implementation on a grain of sand," *IEE Proceedings-Information Security*, vol. 152, no. 1, pp. 13–20, 2005.
- [60] P. Hamalainen, T. Alho, M. Hannikainen, and T. D. Hamalainen, "Design and implementation of low-area and low-power aes encryption hardware core," in *9th EUROMICRO conference on digital system design (DSD'06)*. IEEE, 2006, pp. 577–583.
- [61] T. Good and M. Benaissa, "692-nw advanced encryption standard (aes) on a 0.13- μ m cmos," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 18, no. 12, pp. 1753–1757, 2009.
- [62] F. Haghighizadeh, H. Attarzadeh, and M. Sharifkhani, "A compact 8-bit aes crypto-processor," in *2010 Second International Conference on Computer and Network Technology*. IEEE, 2010, pp. 71–75.
- [63] C. Hocquet, D. Kamel, F. Regazzoni, J.-D. Legat, D. Flandre, D. Bol, and F.-X. Standaert, "Harvesting the potential of nano-cmos for lightweight cryptography: an ultra-low-voltage 65 nm aes coprocessor for passive rfid tags," *Journal of Cryptographic Engineering*, vol. 1, pp. 79–86, 2011.
- [64] M.-C. Chen and W.-T. Li, "An 8-bit rom-free aes design for low-cost applications," *Mathematical Problems in Engineering*, vol. 2013, no. 1, p. 696717, 2013.
- [65] C.-W. Huang, S.-W. Kuo, and C.-J. Chang, "Embedded 8-bit aes in wireless bluetooth application," in *2013 International Conference on System Science and Engineering (ICSSSE)*. IEEE, 2013, pp. 87–92.
- [66] S. Mathew, S. Satpathy, V. Suresh, M. Anders, H. Kaul, A. Agarwal, S. Hsu, G. Chen, and R. Krishnamurthy, "340 mv–1.1 v, 289 gbps/w, 2090-gate nanoaes hardware accelerator with area-optimized encrypt/decrypt gf (2 4) 2 polynomials in 22 nm tri-gate cmos," *IEEE Journal of Solid-State Circuits*, vol. 50, no. 4, pp. 1048–1058, 2015.
- [67] S. Banik, A. Bogdanov, and F. Regazzoni, "Atomic-aes: A compact implementation of the aes encryption/decryption core," in *Progress in Cryptology–INDOCRYPT 2016: 17th International Conference on Cryptology in India, Kolkata, India, December 11-14, 2016, Proceedings 17*. Springer, 2016, pp. 173–190.
- [68] —, "Atomic-aes: A compact implementation of the aes encryption/decryption core," in *Progress in Cryptology–INDOCRYPT 2016: 17th International Conference on Cryptology in India, Kolkata, India, December 11-14, 2016, Proceedings 17*. Springer, 2016, pp. 173–190.
- [69] M. S. Wamser and G. Sigl, "Pushing the limits further: Sub-atomic aes," in *VLSI-SoC: Opportunities and Challenges Beyond the Internet of Things: 25th IFIP WG 10.5/IEEE International Conference on Very Large Scale Integration, VLSI-SoC 2017, Abu Dhabi, United Arab Emirates, October 23–25, 2017, Revised and Extended Selected Papers 24*. Springer, 2019, pp. 220–239.
- [70] J. Yu, "Area and energy optimizations in ASIC implementations of AES and present block ciphers," Master's thesis, University of Waterloo, 2020.
- [71] V. Nandan and R. G. S. Rao, "Minimization of digital logic gates and ultra-low power AES encryption core in 180nm technology," *Microprocessors and Microsystems*, vol. 74, p. 103000, 2020.
- [72] N. Ahmad and S. Hasan, "A new ASIC implementation of an advanced encryption standard (AES) crypto-hardware accelerator," *Microelectron. J.*, vol. 117, no. C, Nov. 2021. [Online]. Available: <https://doi.org/10.1016/j.mejo.2021.105255>
- [73] M. Gazzaro and J. P. Carmo, "Power consumption efficiency of encryption schemes for RFID," *Chips*, vol. 3, no. 3, pp. 216–228, Jul. 2024.
- [74] A. Stillmaker and B. Baas, "Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm," *Integration, the VLSI Journal*, vol. 58, pp. 74–81, 2017, <http://vcl.ece.ucdavis.edu/pubs/2017.02.VLSIIntegration.TechScale/>.
- [75] S. Sarangi and B. Baas, "DeepScaleTool: A tool for the accurate estimation of technology scaling in the deep-submicron era," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2021, pp. 1–5.